



Foundations: what is Viz?

Luís Paulo Santos

Departamento de Informática

2025



“The purpose of computing is insight not numbers.”

-- R. W. Hamming (1961)

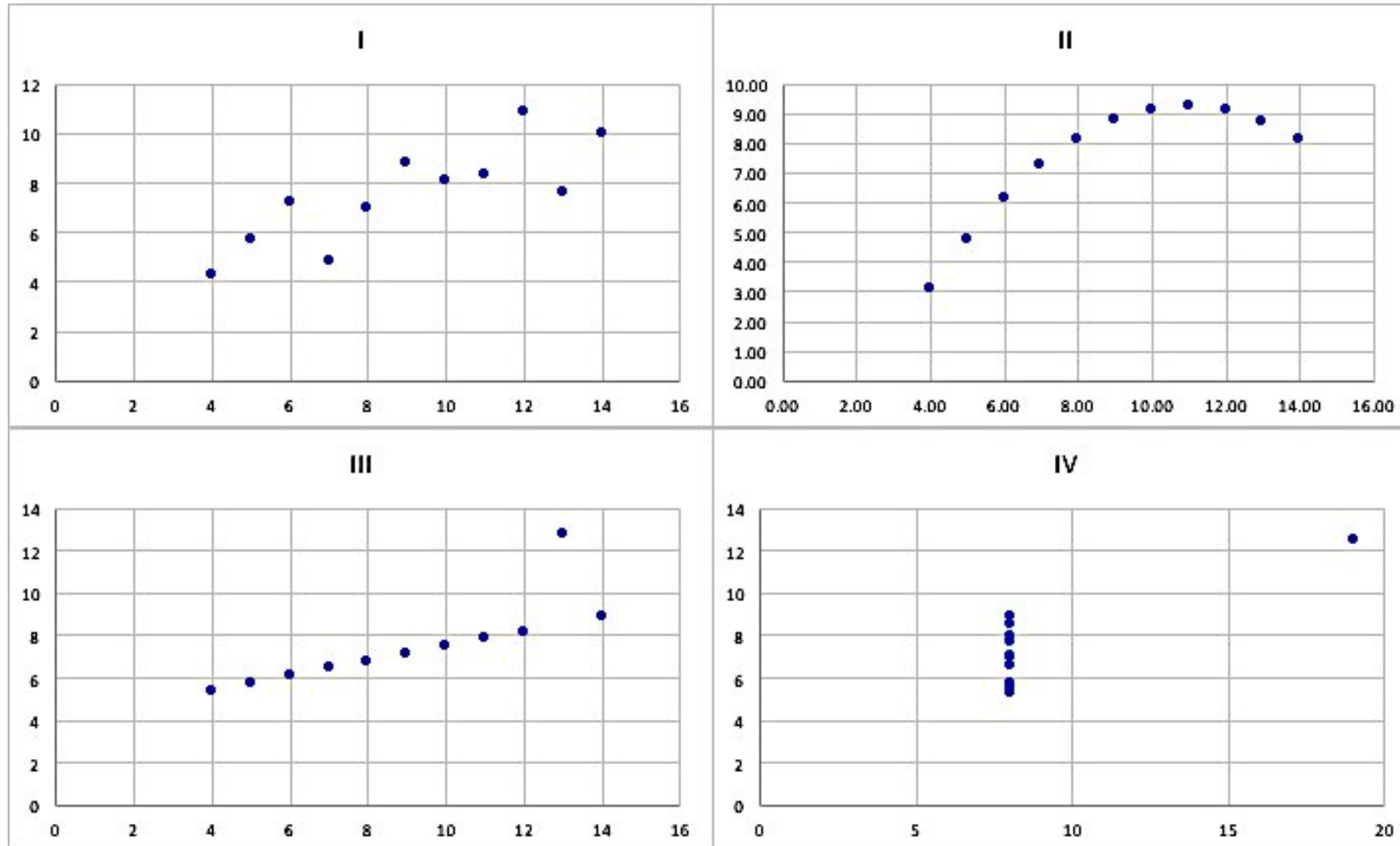


Anscombe's Quartet

	I		II		III		IV	
	x	y	x	y	x	y	x	y
	10.00	8.04	10.00	9.14	10.00	7.46	8.00	6.58
	8.00	6.95	8.00	8.14	8.00	6.77	8.00	5.76
	13.00	7.58	13.00	8.74	13.00	12.74	8.00	7.71
	9.00	8.81	9.00	8.77	9.00	7.11	8.00	8.84
	11.00	8.33	11.00	9.26	11.00	7.81	8.00	8.47
	14.00	9.96	14.00	8.10	14.00	8.84	8.00	7.04
	6.00	7.24	6.00	6.13	6.00	6.08	8.00	5.25
	4.00	4.26	4.00	3.10	4.00	5.39	19.00	12.50
	12.00	10.84	12.00	9.13	12.00	8.15	8.00	5.56
	7.00	4.82	7.00	7.26	7.00	6.42	8.00	7.91
	5.00	5.68	5.00	4.74	5.00	5.73	8.00	6.89
Mean	9.00	7.50	9.00	7.50	9.00	7.50	9.00	7.50
Variance	11.00	4.13	11.00	4.13	11.00	4.13	11.00	4.13
Correlation	0.82		0.82		0.82		0.82	
Regression	$y = 3 + 0.5 x$		$y = 3 + 0.5 x$		$y = 3 + 0.5 x$		$y = 3 + 0.5 x$	

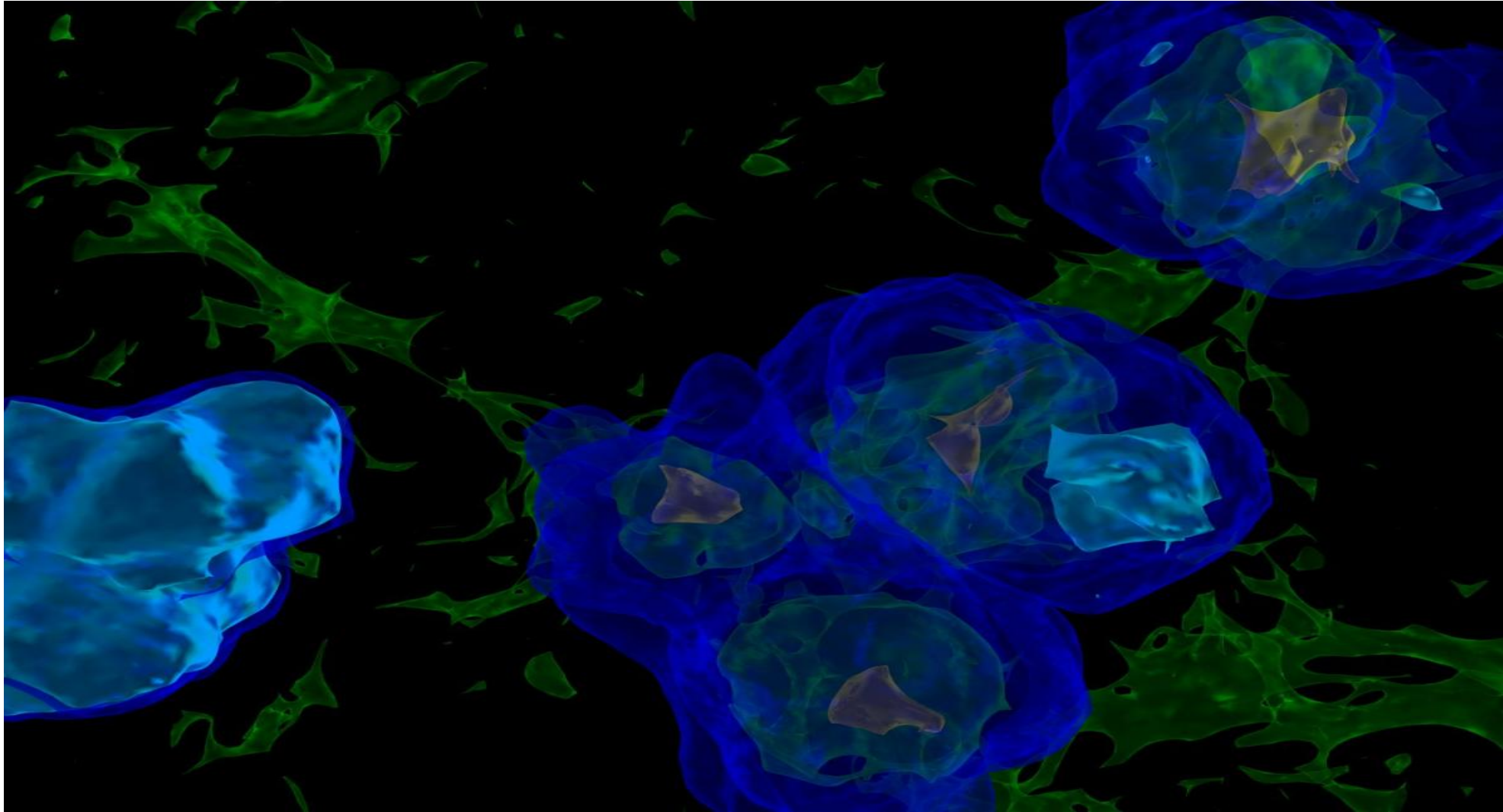


Anscombe's Quartet





Raw Data versus Visualization

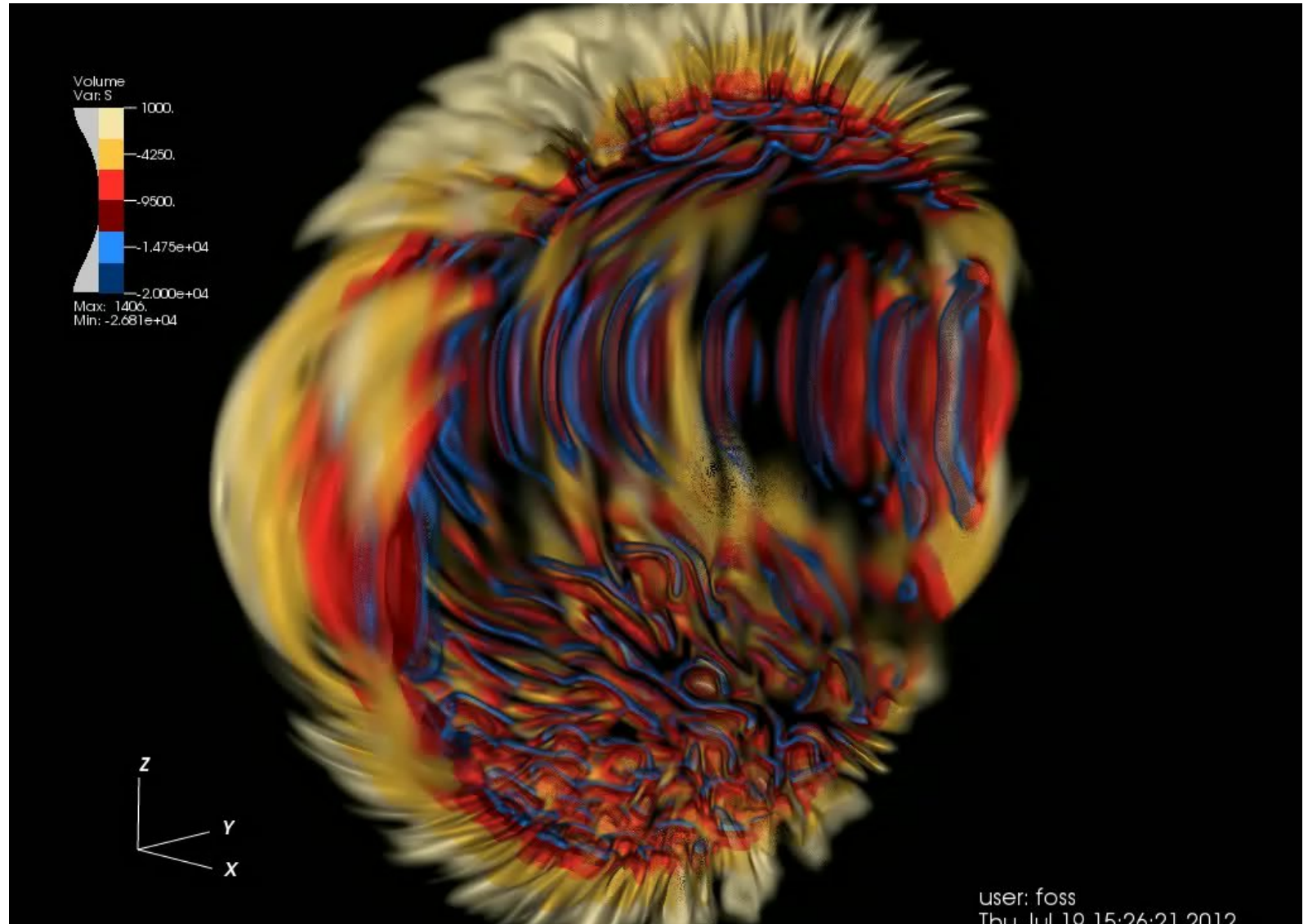


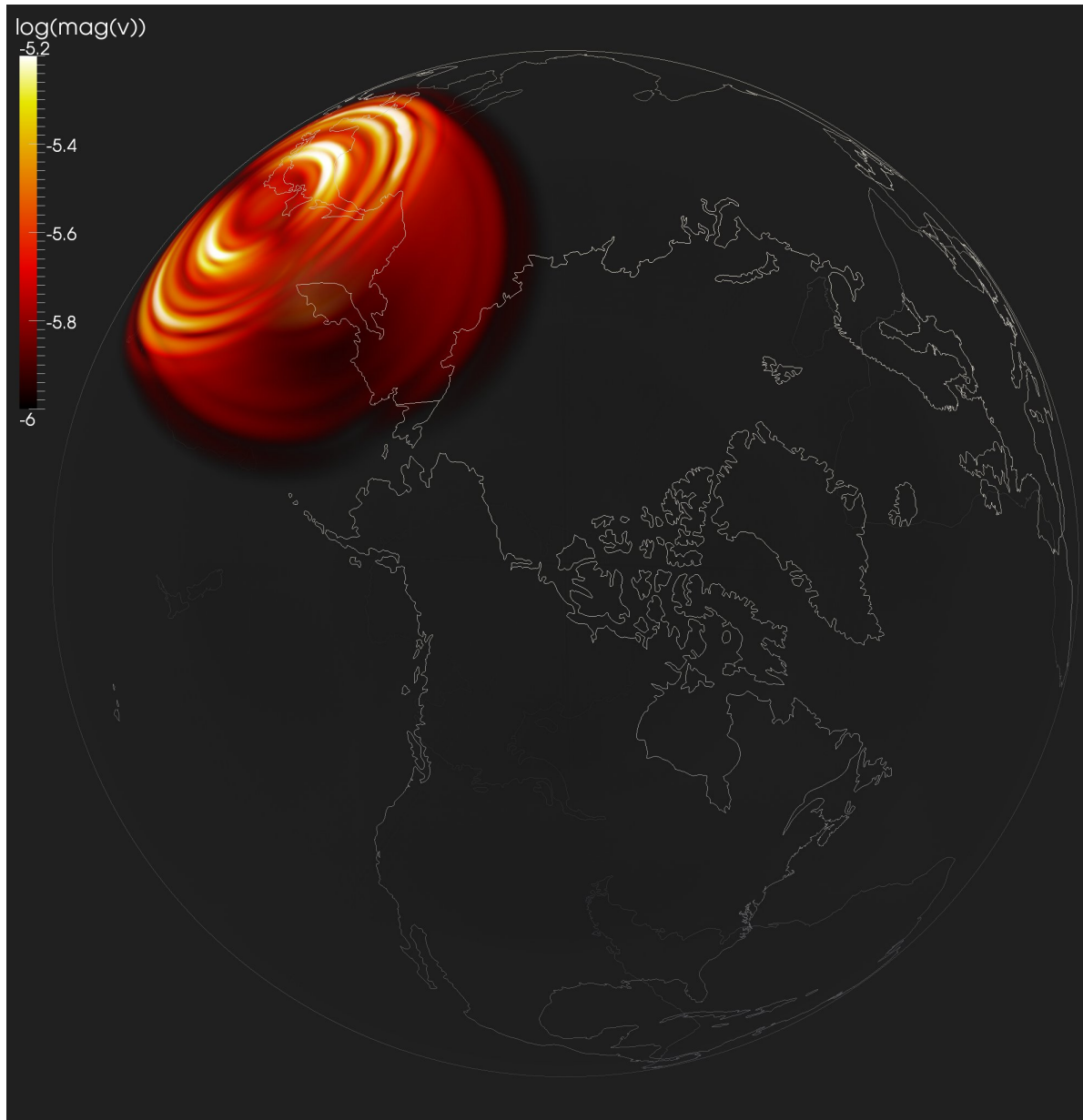
Example – Stellar Magnetism

- A Sun-like star undergoes magnetic cyclic reversal, shown by field lines.
- Terabytes of data to mine through and visualize.

Greg Foss, TACC;

*Ben Brown, University of Wisconsin,
Madison*





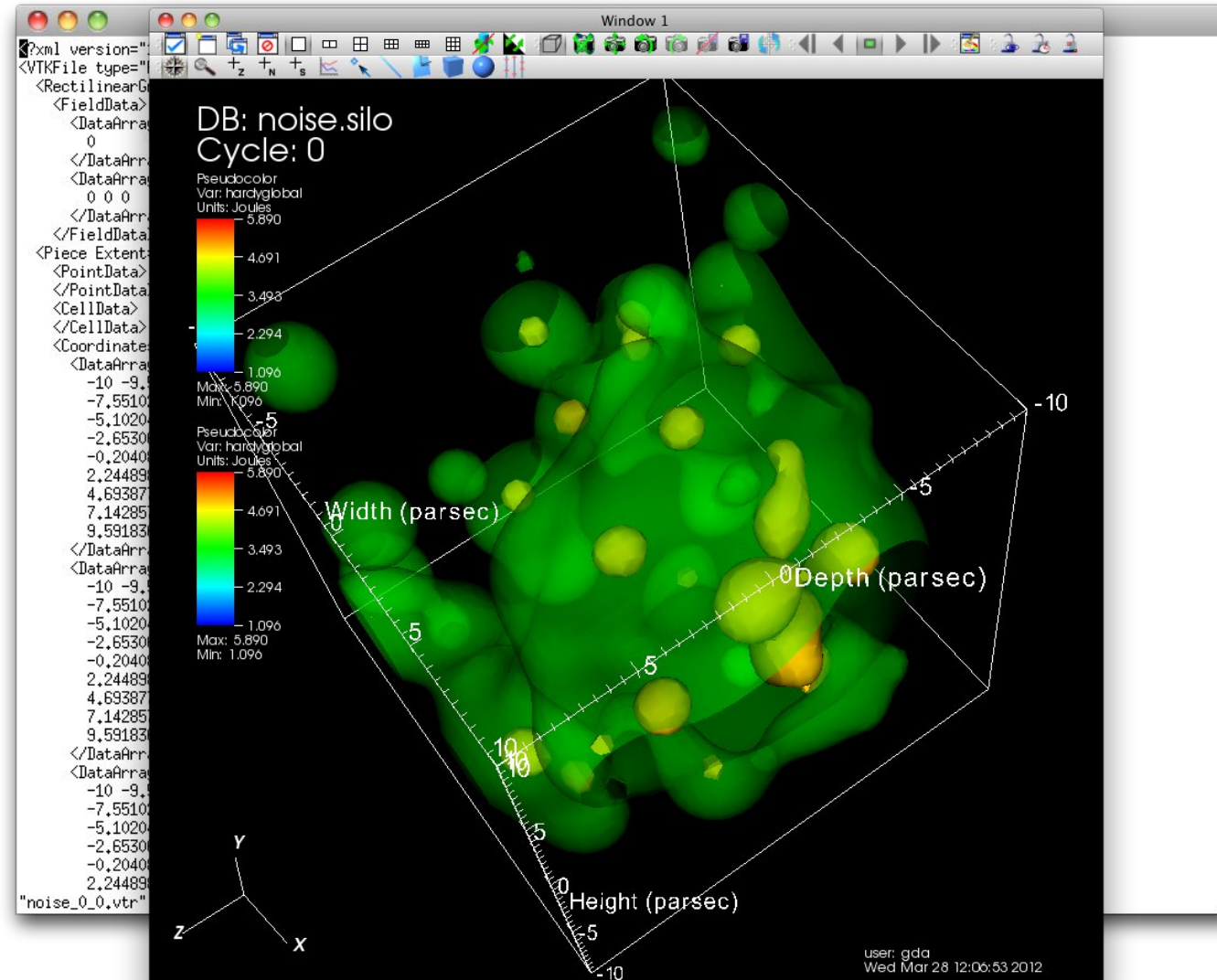
Tera-Scale Global Seismic Wave Propagation

- Visualization corresponds to a simulation of global wave propagation from a simplified model of the 2011 Tohoku earthquake with a central source frequency of 1/85 Hz, using 93 million unknowns on TACC's Lonestar system

Carsten Burstedde, Omar Ghattas, James Martin, Georg Stadler and Lucas Wilcox, ICES; Greg Abram, TACC



Raw Data versus Visualization



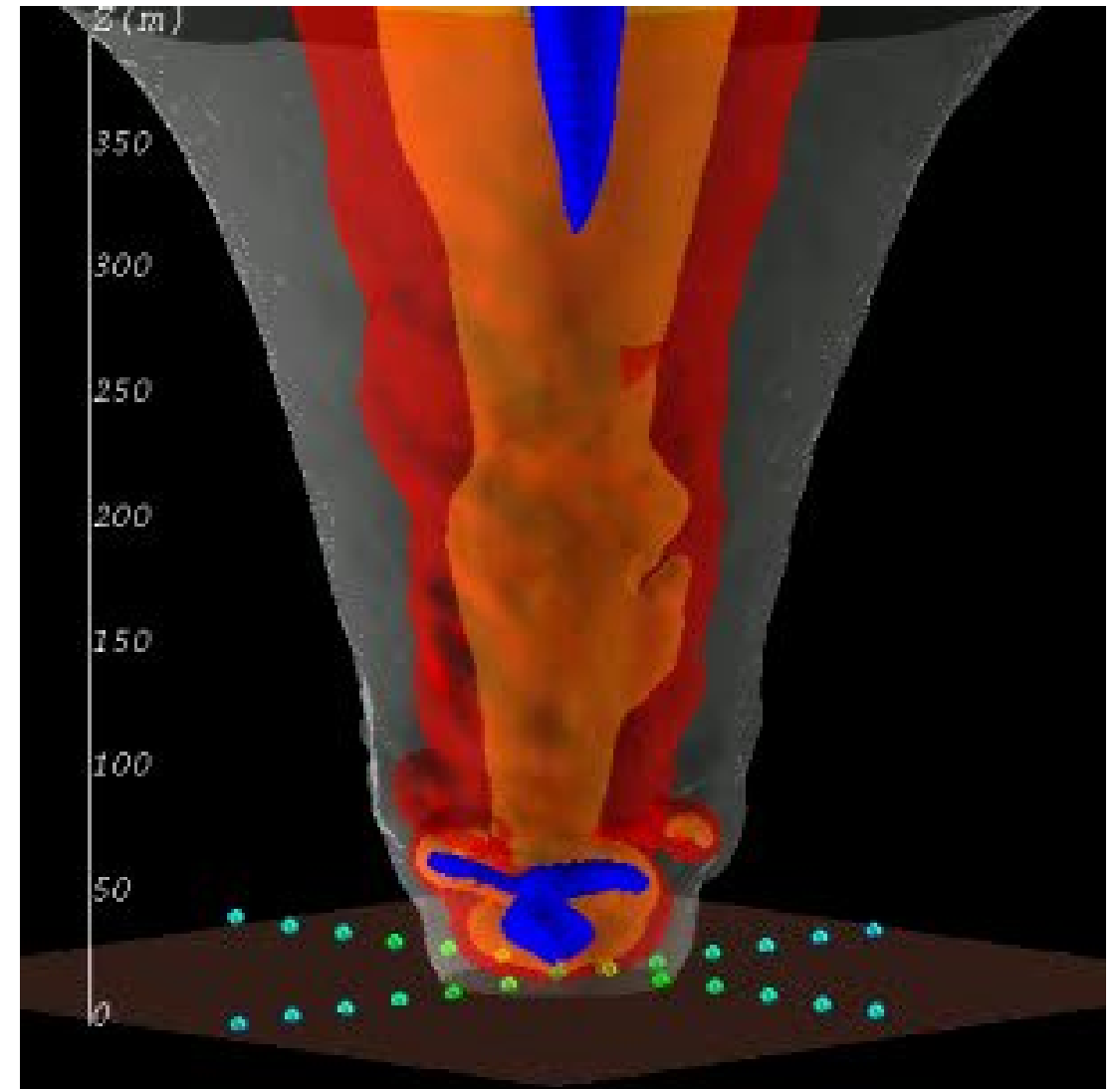
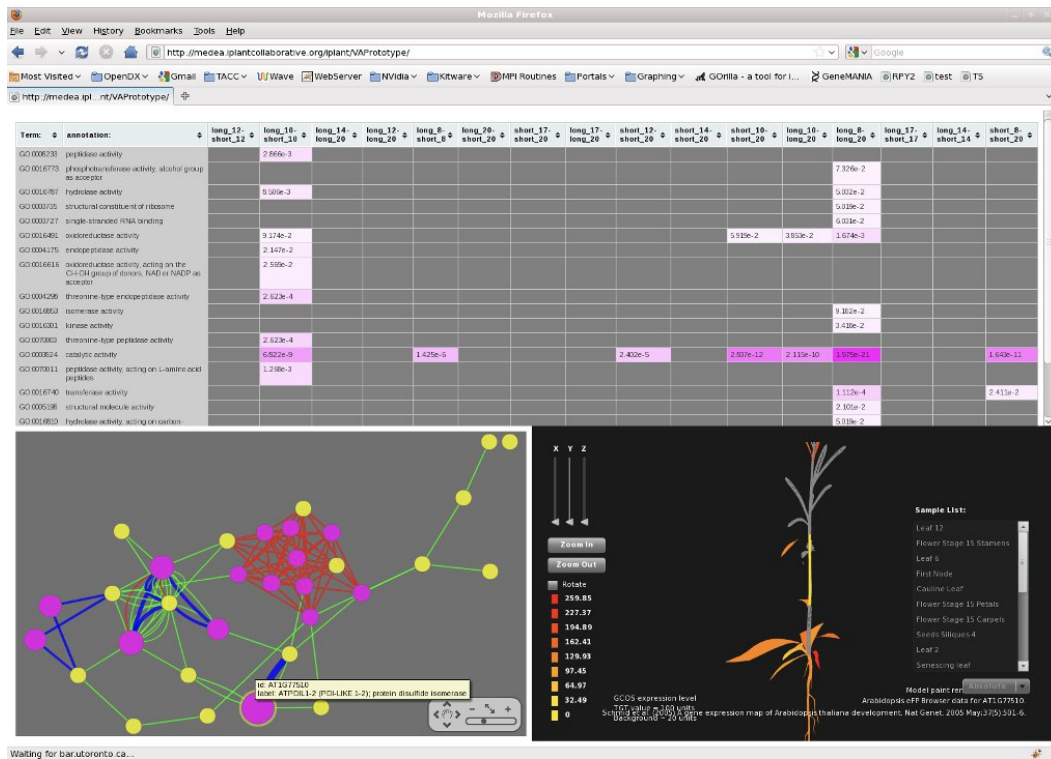


	SciViz	InfoViz
Data type	Physically based, spatial, and continuous data (e.g., weather simulations, MRI scans)	Abstract, non-physical, and often discrete data (e.g., financial stocks, organizational charts, social networks)
Goal	To provide insight into physical phenomena and help understand complex, geometry-based data	To represent abstract data visually to help people understand complex relationships and patterns
Supportig Disciplines	Often involves computer graphics, geometry, and numerical computation	Often involves human-computer interaction, psychology (human perception), and statistics
Examples	Visualizing the flow of a hurricane, a CT scan, or a computational fluid dynamics (CFD) simulation	Creating bar charts, pie charts, heatmaps, or network diagrams

Note: many see this more as a community divide than as a fundamental, ontological divide

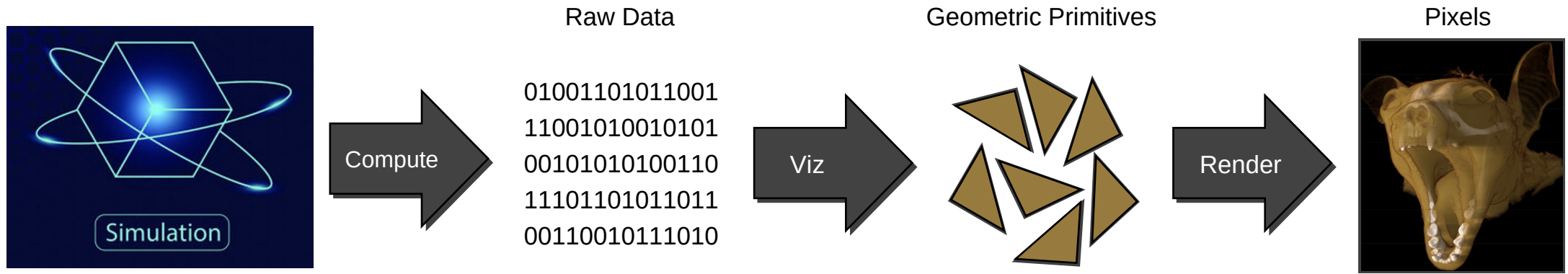
[“SciVis, InfoVis– bridging the community divide?!”; Daniel Weiskopf , Kwan-Liu Ma, Jarke J. van Wijk, Helwig Hauser]

SciViz and InfoViz



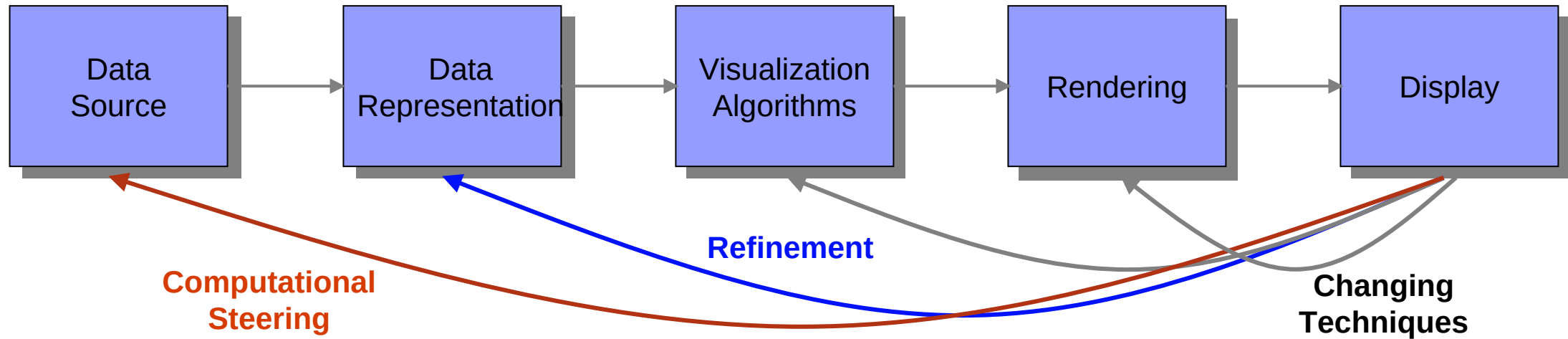


Viz: “seeing” the Science





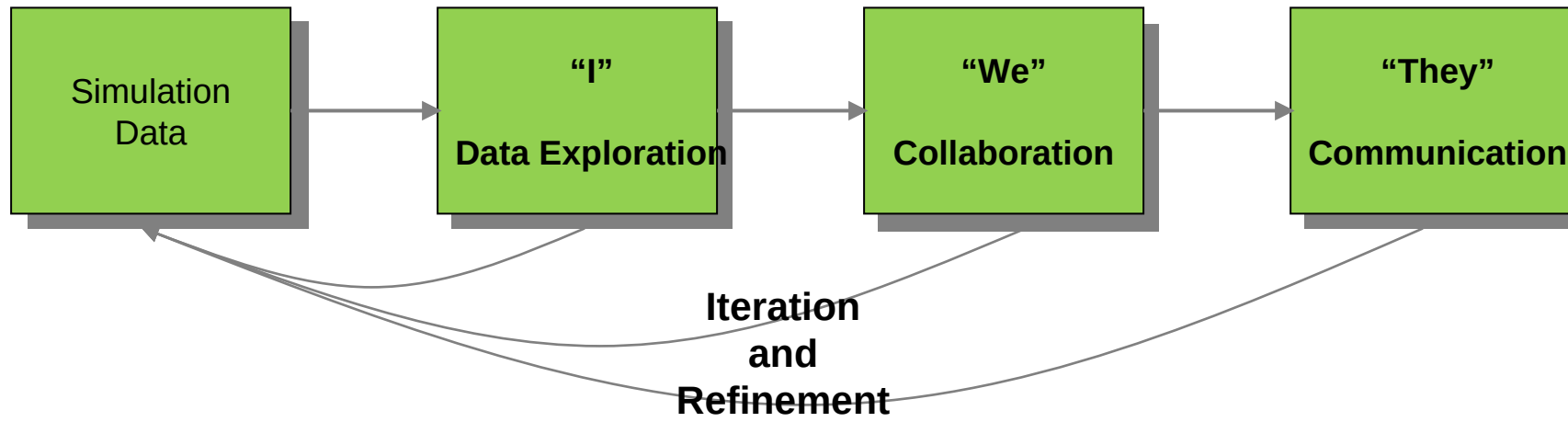
Getting from Data to Insight



... using insight to get more insight



“I, We, They” Development Path



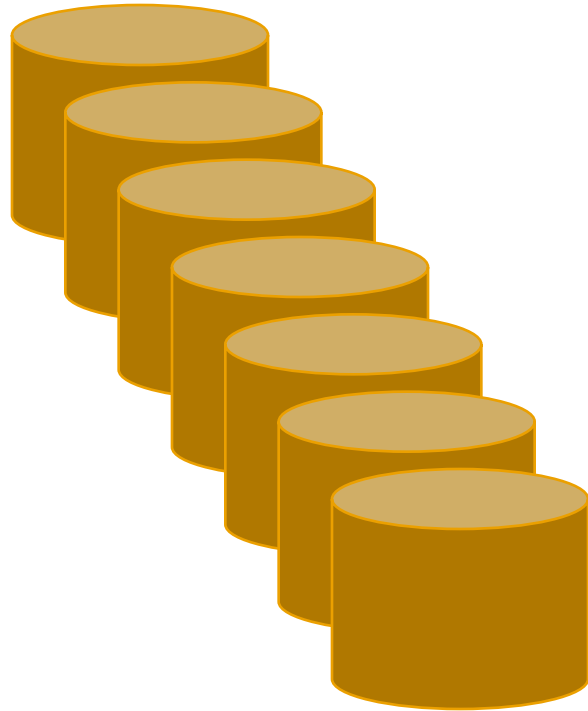


SciViz Data Domain Dimensionality

- Data mapped onto a 1 to 4D computational domain (e.g. heat distribution in a 3D block of material)
 - $T(x,y,z)$ – temperature at point (x,y,z) within the block
- The data can be multivariate:
 - $T(x,y,z)$ – temperature at point (x,y,z) within the block
 - $P(x,y,z)$ - pressure at point (x,y,z) within the block
 - $M(x,y,z)$ – type of material at point (x,y,z) within the block (eventually a non-numerical label)
- Time is the most common 4th dimension ((e.g. heat transfer in a 3D block of material)
 - $T(x,y,z,t)$ – temperature at point (x,y,z) within the block at instant t



Higher dimensional data domain



An Oil Tank Farm

$$\text{state} = (l_0, l_1, l_2, l_3, l_4, l_5, l_6)$$

A 7D space representing *all possible states* of a 7 tank system

$$\text{Content}(l_0, l_1, l_2, l_3, l_4, l_5, l_6) = l_0 + l_1 + l_2 + l_3 + l_4 + l_5 + l_6$$

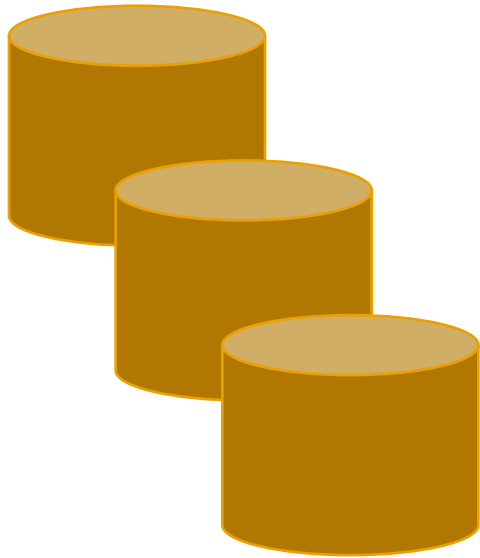
A *function* on 7-space representing the amount of oil in the tank farm



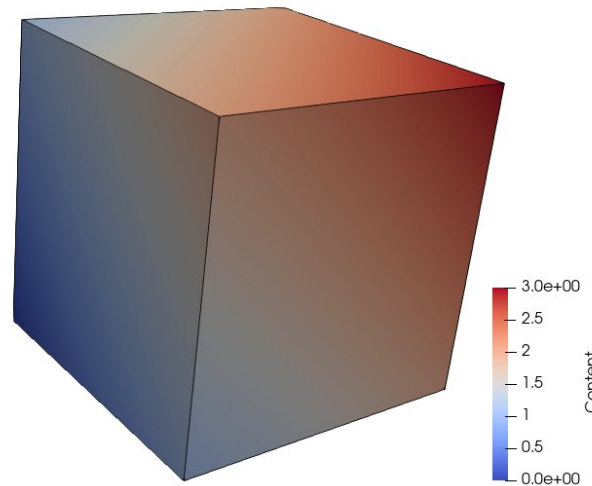
High dimensional data domain

Consider 3 tank farm....

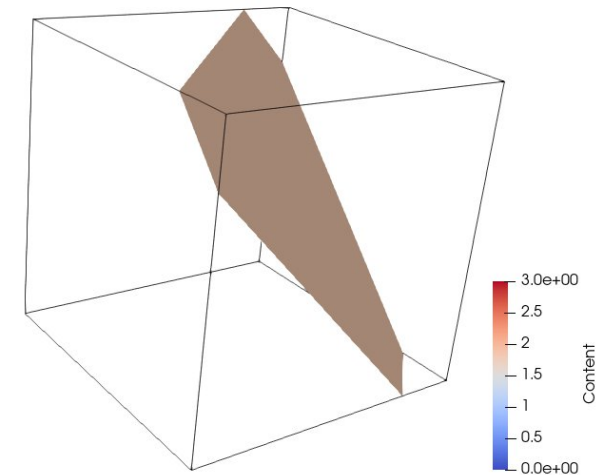
$state = (l_0, l_1, l_2, l_3, l_4, l_5, l_6)$



An Oil Tank Farm



Content (i,j,k)
colormapped



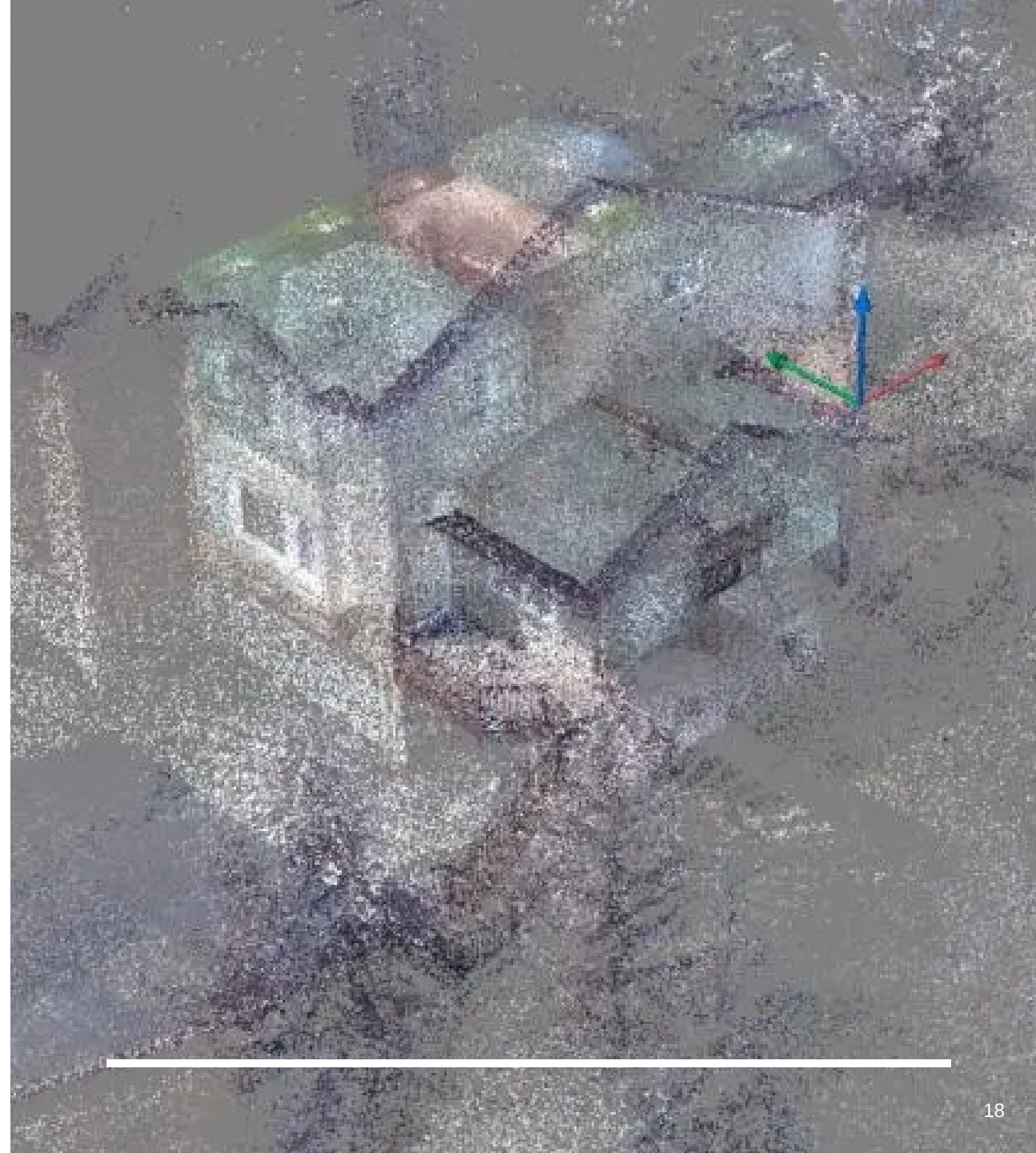
(i,j,k) such that
Content(i,j,k) = 1.75



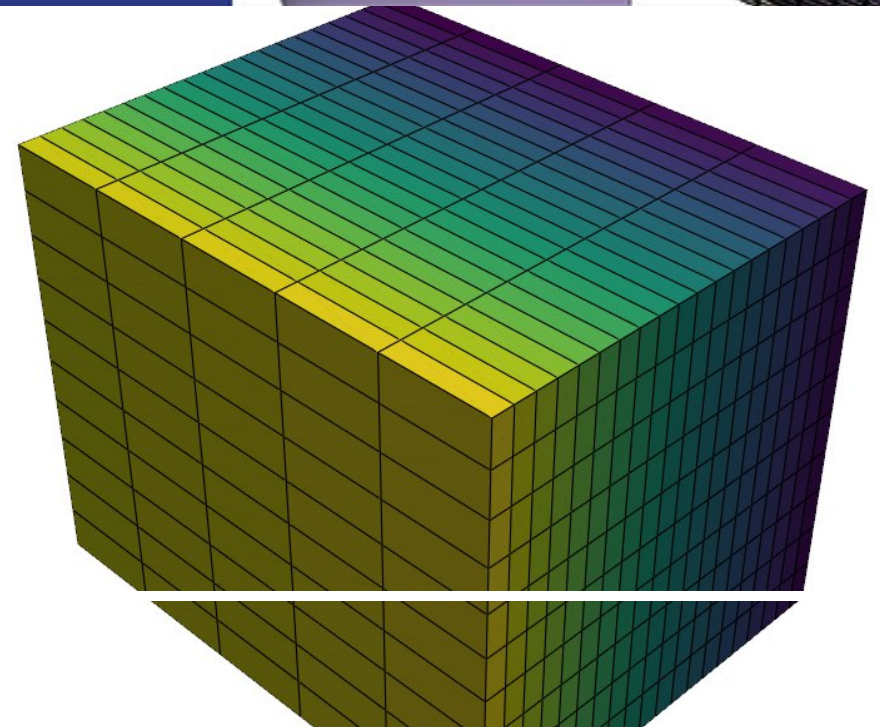
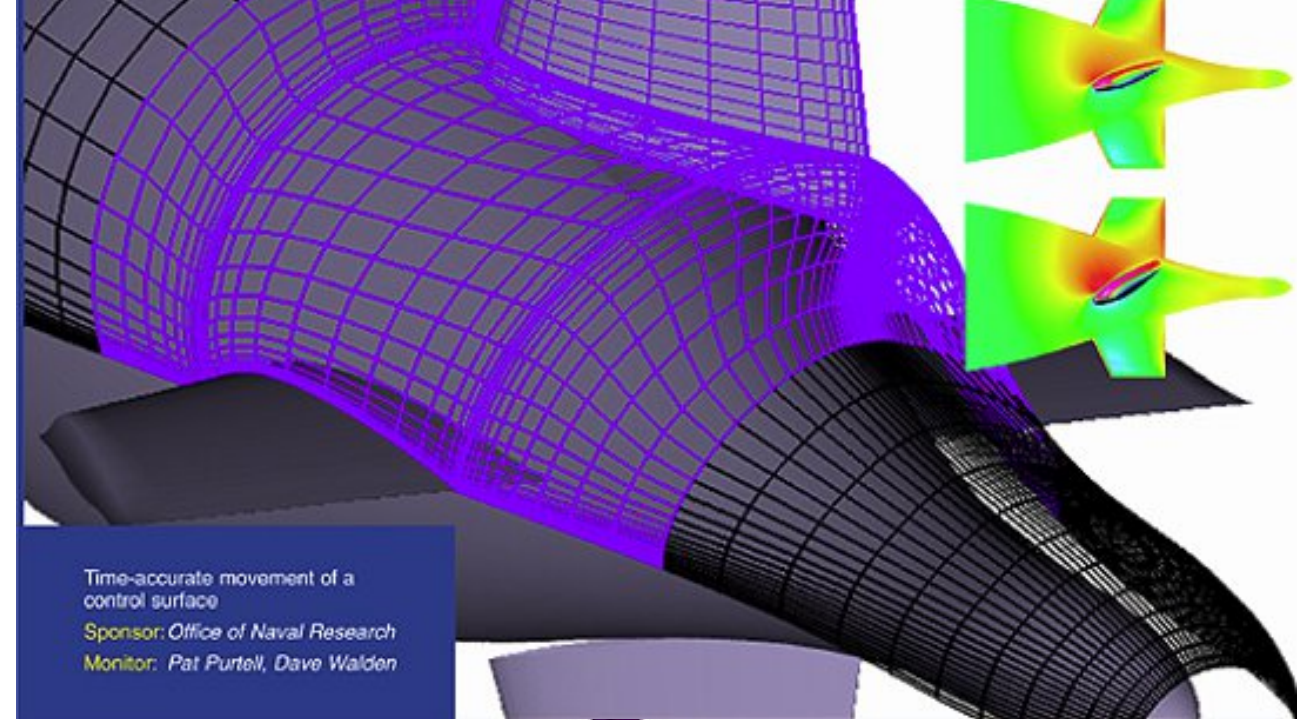
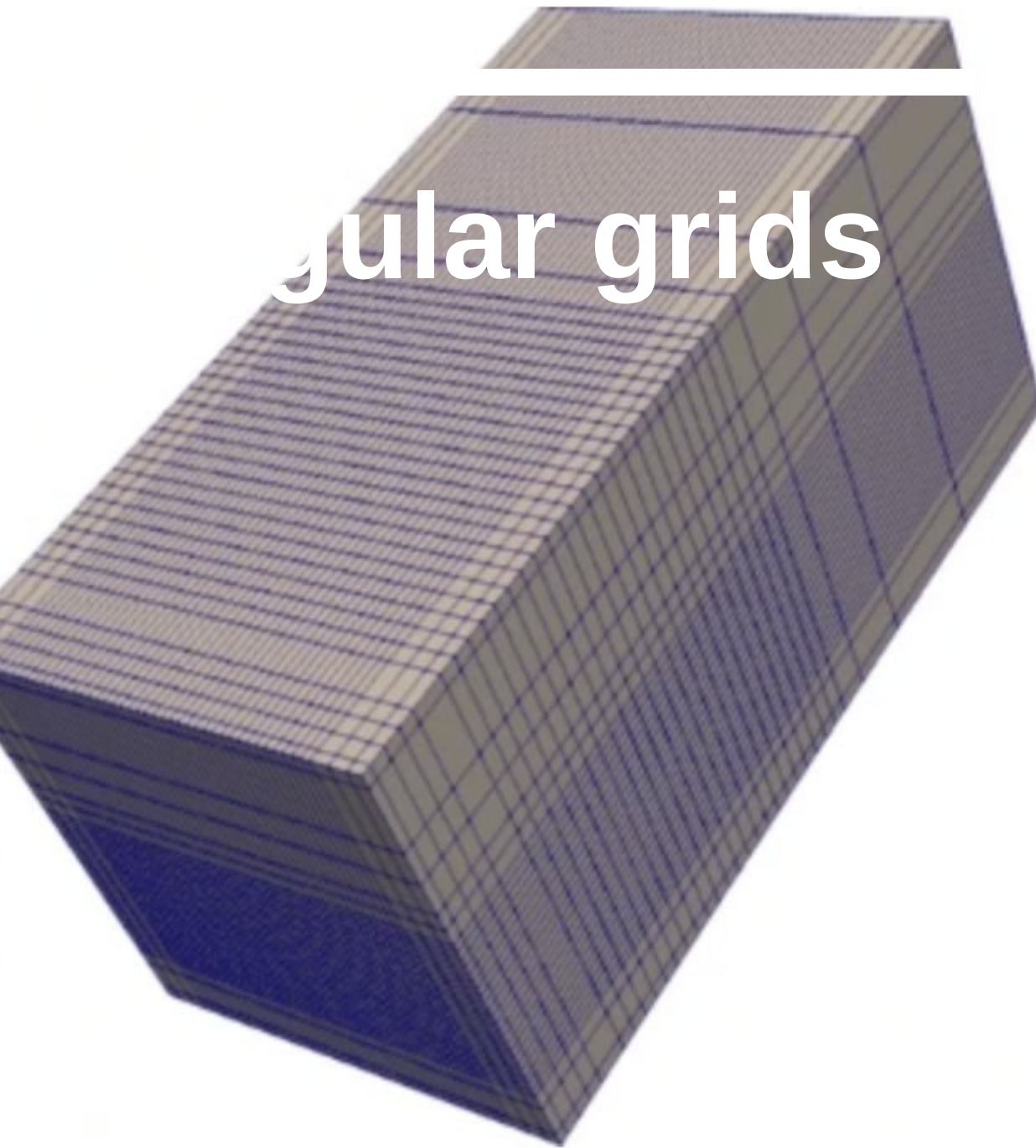
Types of input data

- The attributes (e.g., temperature) cannot be specified at the infinite many points of a continuous problem domain
- The problem domain is therefore discretized into a finite number of points, values elsewhere are interpolated
- Structure of the data set:
 - **Point cloud / particles** – scattered values with no defined structure (e.g. laser scan / n-body simulation)
 - **Regular grid** - regular structure, all voxels (cells) are the same size and shape (e.g. medical scan)
 - **Rectilinear grid** - regular grid, but spacing can vary across each axis
 - **Curvilinear grid** - regular grid shaping function applied (e.g. engineering model)
 - **Unstructured / irregular** - irregular mesh (e.g. extracted surface)
- Within grids the following are identified:
 - **Vertices / Nodes** – finite number of points in the computational space
 - **Cells** – join vertices into space-filling interpolation elements
 - **Data** – values associated with each vertex or with each cell

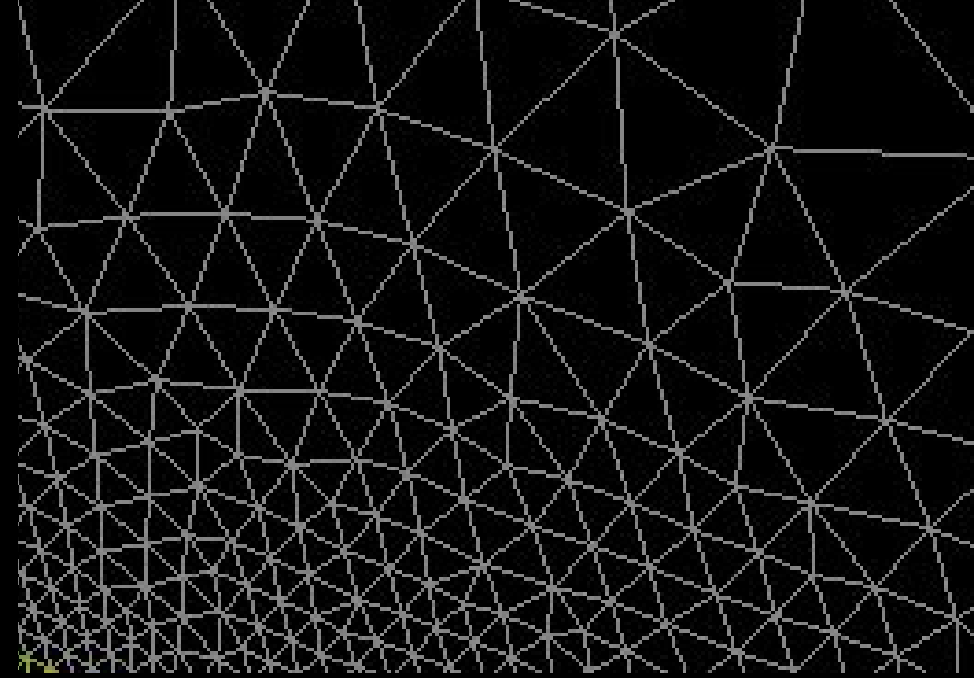
Point clouds



gular grids

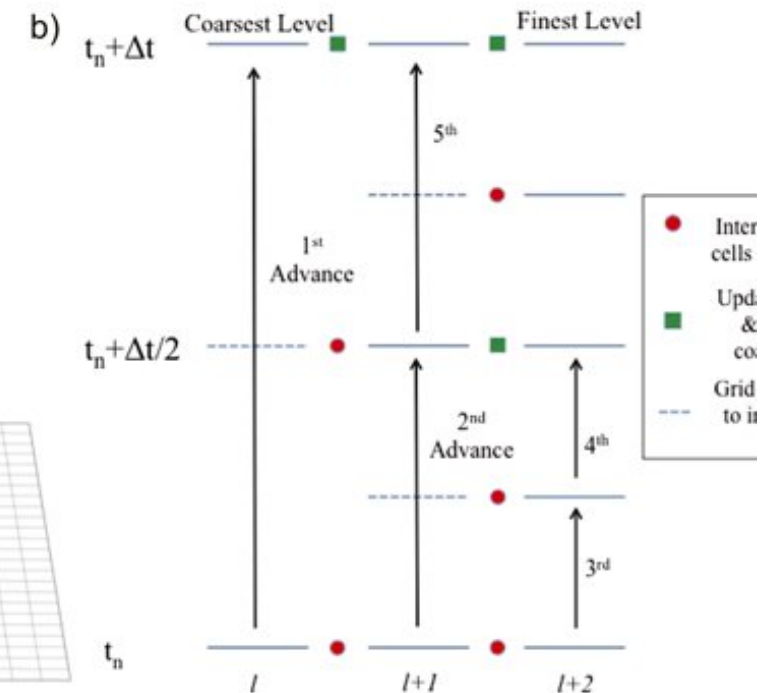
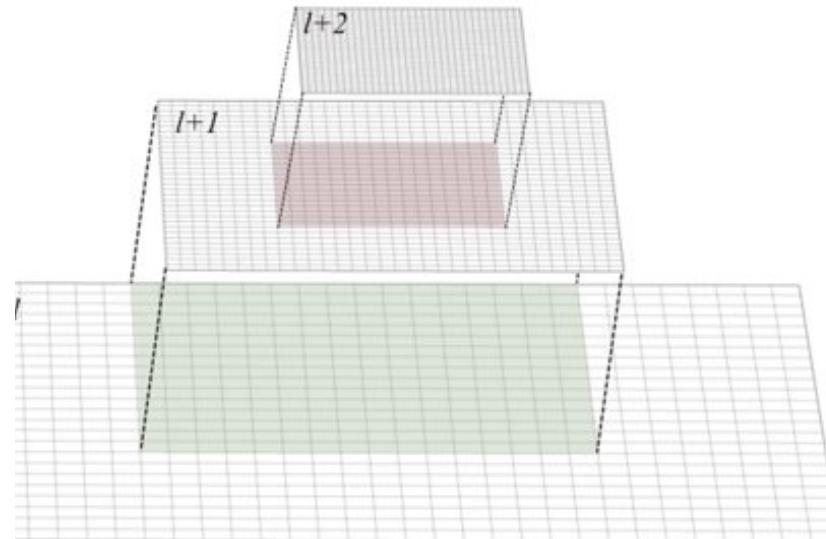


unstructured grids



Adaptive Refinement

- Use efficient representations (eg. rectilinear grids) but insert higher resolution grids **where and when** necessary
- Kind of best of all worlds, but awkward to avoid discontinuities at mesh boundaries





- Scalar fields

- the data at each point $\dot{p}$ of a d-dimensional domain is a scalar value (magnitude):

$$\text{temperature}(\dot{p}) = t, \quad t \in \mathbb{R}^+$$

- Vector fields

- the data at each point $\dot{p}$ of a d-dimensional domain is a vector:

$$\text{velocity}(\dot{p}) = \{v_x, v_y, v_z\}, \quad v_i \in \mathbb{R}$$

$$\nabla \text{temperature}(\dot{p}) = \left\{ d_t/d_x, d_t/d_y, d_t/d_z \right\}, \quad d_t/d_i \in \mathbb{R}$$

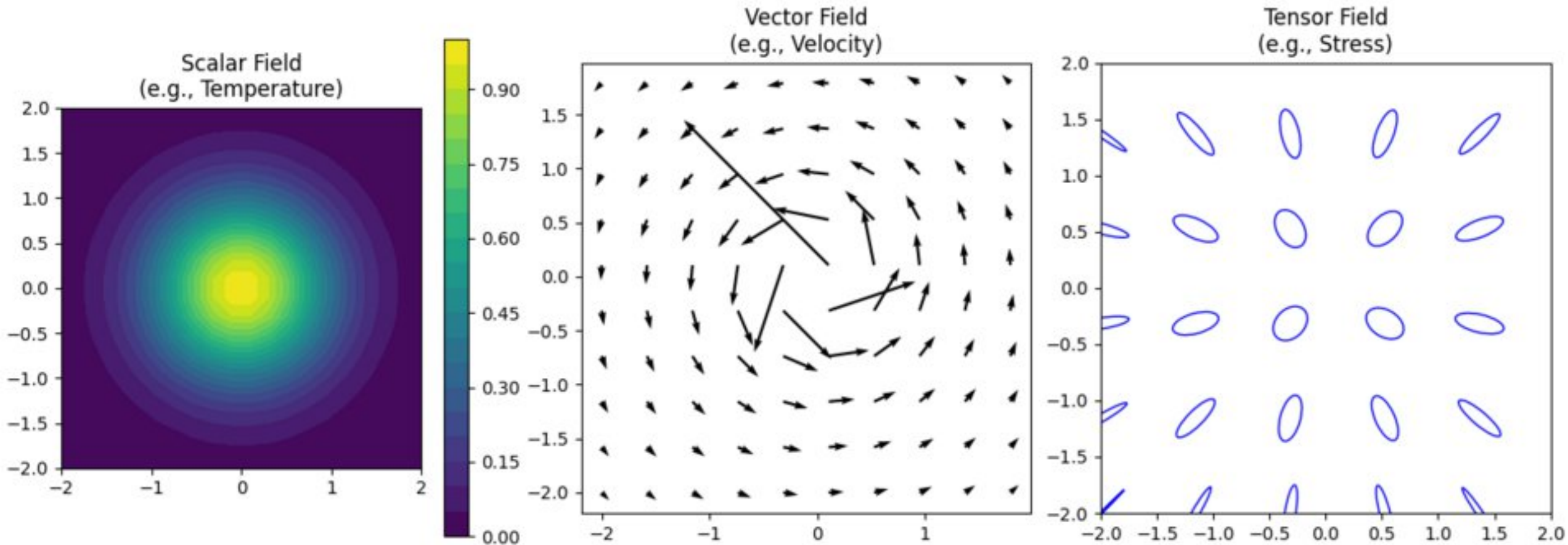
- Tensor fields

- the data at each point $\dot{p}$ of a d-dimensional domain is a d-dimensional tensor (generalization of vectors and matrices):

$$\text{stressTensor}(\dot{p}) = \begin{bmatrix} \sigma_x & \tau_{xy} & \tau_{xz} \\ \tau_{yx} & \sigma_y & \tau_{yz} \\ \tau_{zx} & \tau_{zy} & \sigma_z \end{bmatrix}$$

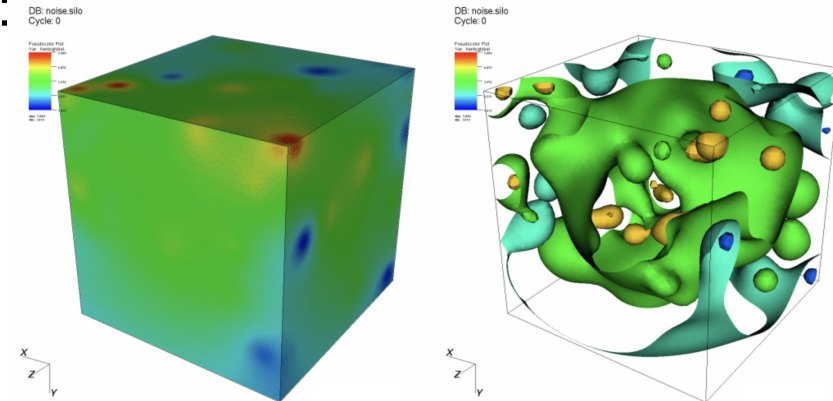


Visualization: data types



- How do we go from the data set original values to optical properties that can be visualized?
- The transfer function assigns optical properties (e.g., RGB colour and opacity) to each data point as a function of the local or global data values:

$$T(\dot{p}, \mathcal{D}) \rightarrow (R, G, B, \alpha)$$



- Example: by setting opacity to 1 for data points that comply with some criterium and $\alpha = 0$ for all remaining points an isosurface can be visualized within a 3D data set

$$T(\dot{p}, \mathcal{D}) = \begin{cases} (R, G, B) = cmap(f_{RGB}(\dot{p})), \alpha = 1 & \leftarrow k - \varepsilon < f(\dot{p}, \mathcal{D}) < k + \varepsilon \\ (R, G, B) = (0, 0, 0), \alpha = 0 & \leftarrow \neg \end{cases}$$

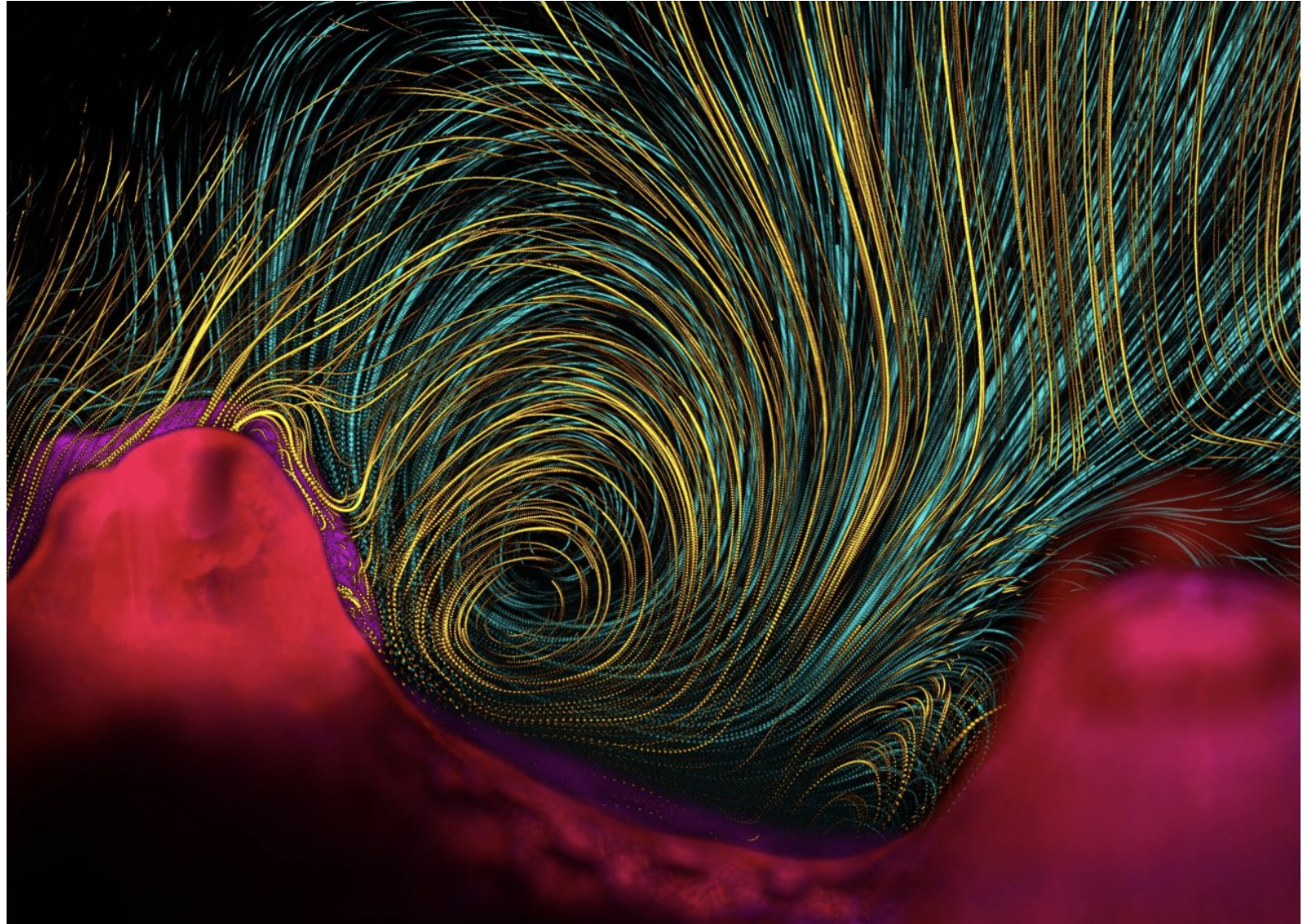


- The d-dimensional discrete data set specifies the values of relevant properties at well defined locations in the domain.
- For example, for a 3D geometric space $f(x,y,z)$ is known for points (x,y,z) corresponding to a grid node.
- Estimating the data value at off-grid points requires **interpolation** from the known values $\mathcal{D}$

0-order (nearest neighbour)	1 st order ((bi/tri)-linear)
$f(x,y,z) = \mathcal{D}(\text{round}(x), \text{round}(y), \text{round}(z))$	(x_0, y_0, z_0) - grid node with minimum coordinates for the cell containing (x, y, z) (x_1, y_1, z_1) - grid node with maximum coordinates for the cell containing (x, y, z) (u, v, w) - normalized distance (x, y, z) to (x_0, y_0, z_0) $\begin{aligned} f(x, y_0, z_0) &= u f(x_0, y_0, z_0) + (1-u) f(x_1, y_0, z_0) \\ f(x, y, z_0) &= v f(x, y_0, z_0) + (1-v) f(x, y_1, z_0) \\ f(x, y, z) &= w f(x, y, z_0) + (1-w) f(x, y, z_1) \end{aligned}$

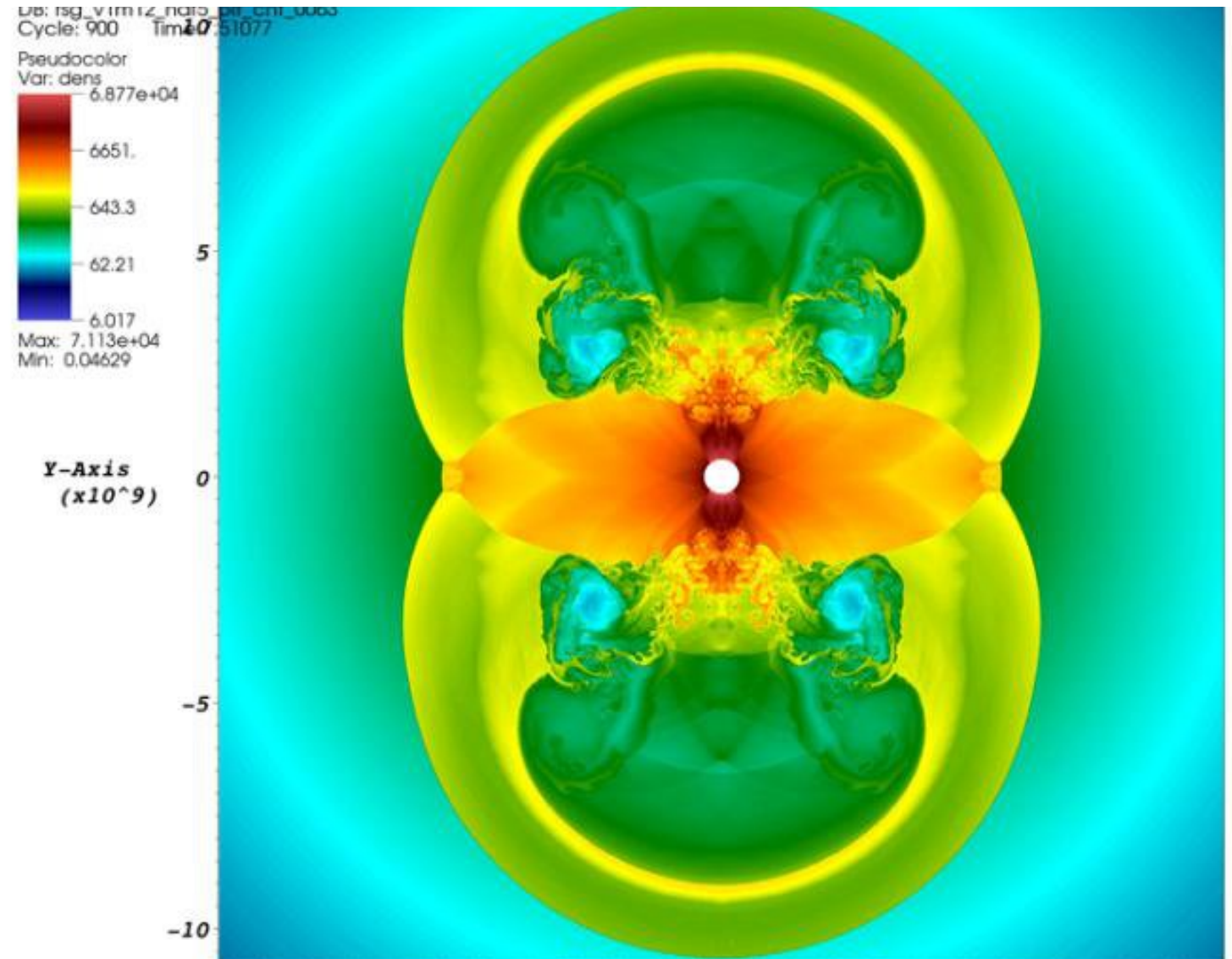
Visualization Operations

- Surface Shading (Pseudocolor)
- Isosurfacing (Contours in 2D)
- Volume Rendering
- Clipping Planes
- Vector Fields
- Streamlines



Surface Shading (Pseudocolor)

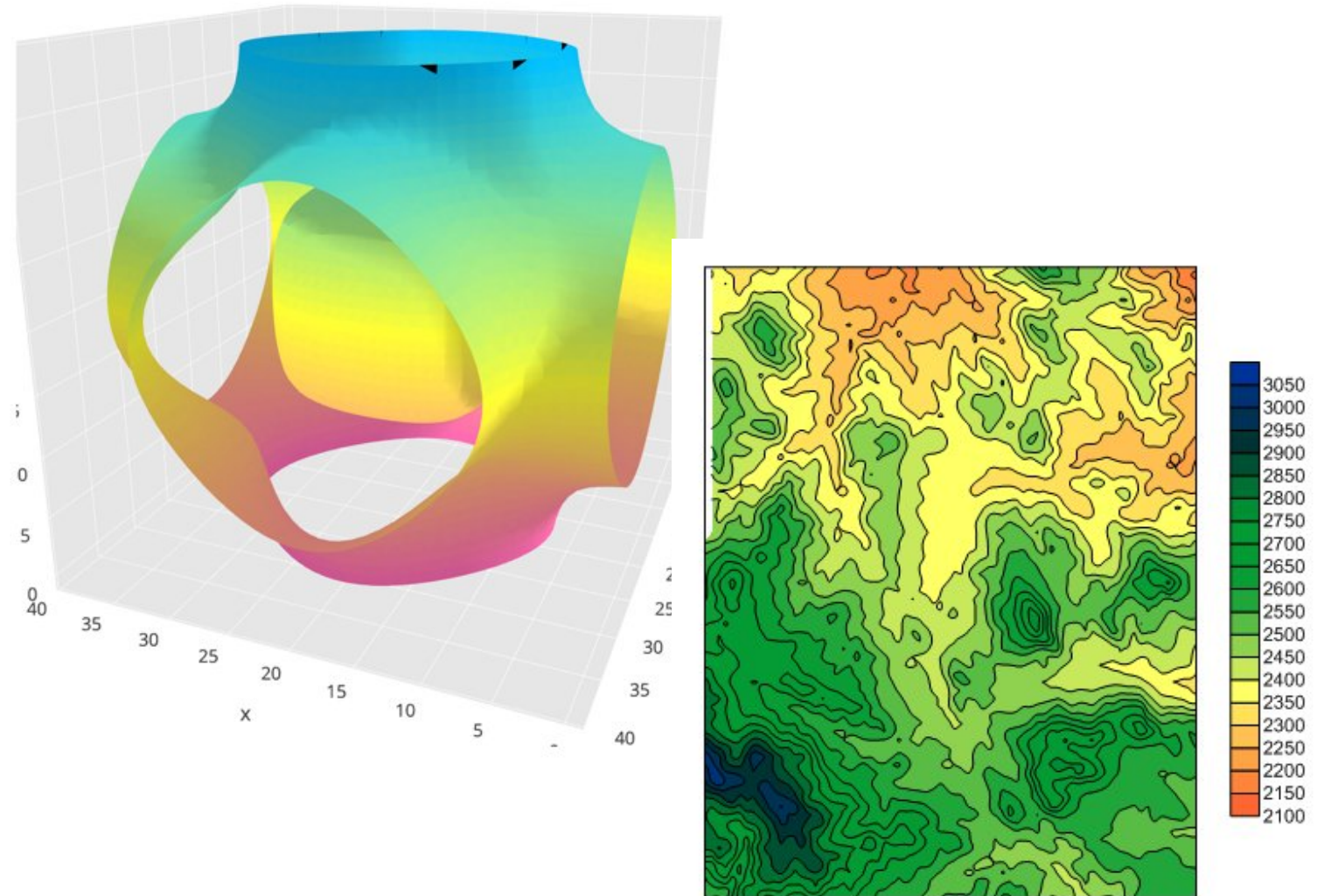
- Given a scalar value at a point on the surface and a color map,
- find the corresponding color (and opacity) and apply it to the surface point.
- Most common operation, often combined with other ops





Isosurfacing (Contours in 2D)

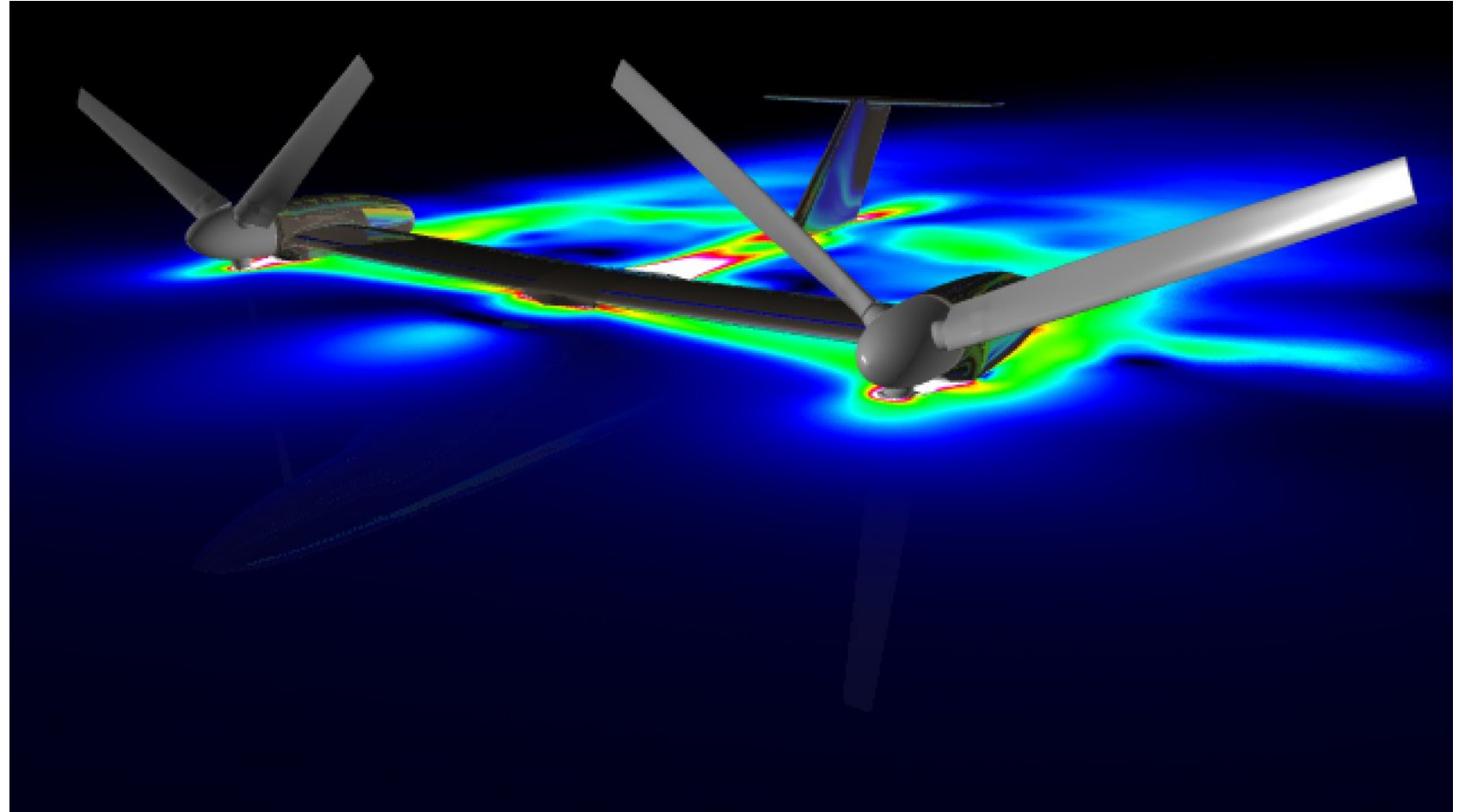
- Surface (2D: line) that represents points of constant value within a volume (2D: surface)
- Plot the surface for a given scalar value.
- Good for showing known values of interest





Clipping Planes

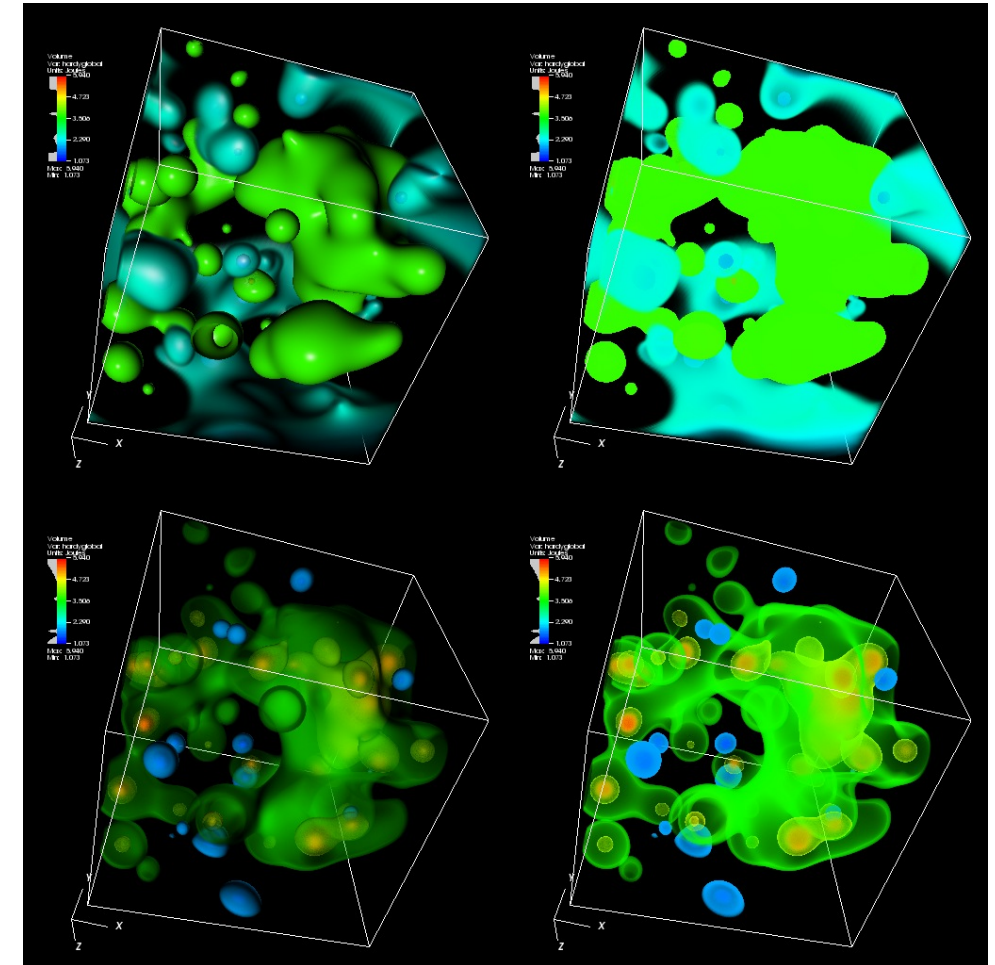
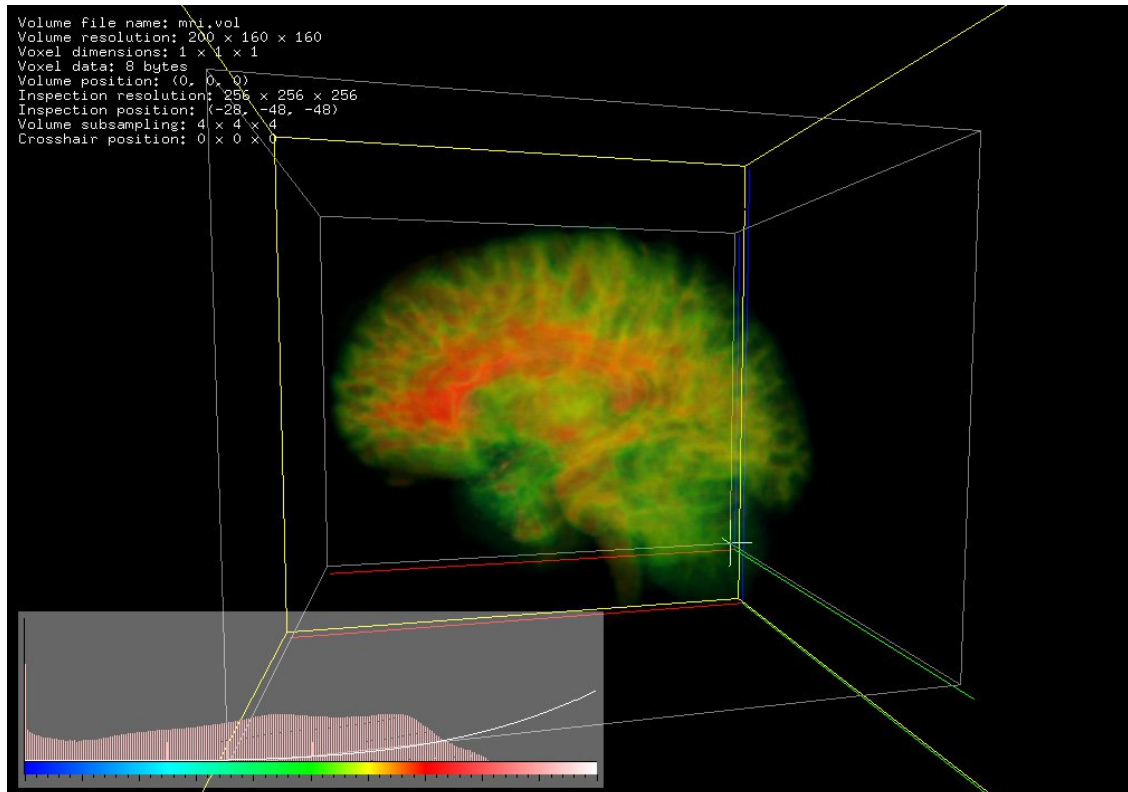
- Extract a plane from the data to show features
- Hide part of dataset to expose features



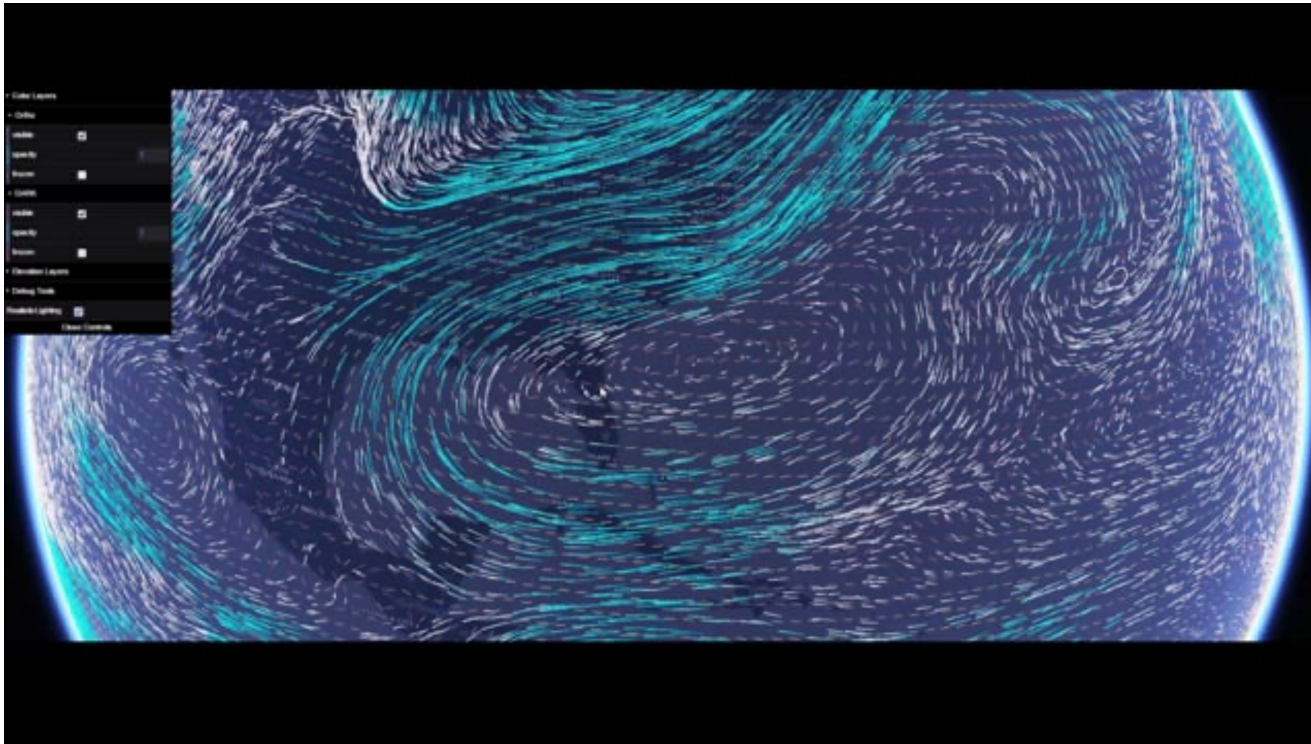


Volume Rendering

- Expresses how light travels through a volume
- Color & opacity controlled by transfer function
- Smoother transitions than isosurfaces

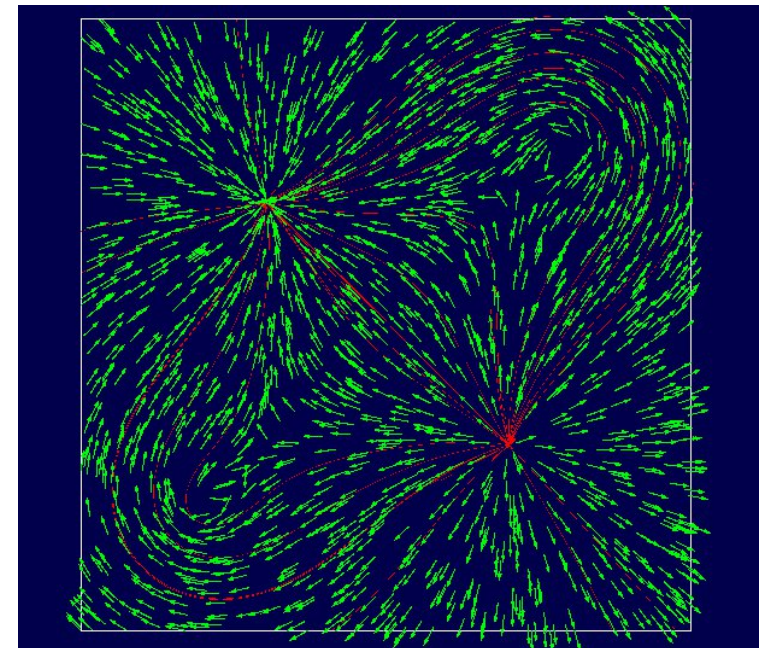


Vector fields



[Alexander Devaux; <https://vimeo.com/216138366>]

- Arrows are drawn at a subset of points in a grid;
- The direction indicates the vector's direction;
- Length and/or color can represent its magnitude.





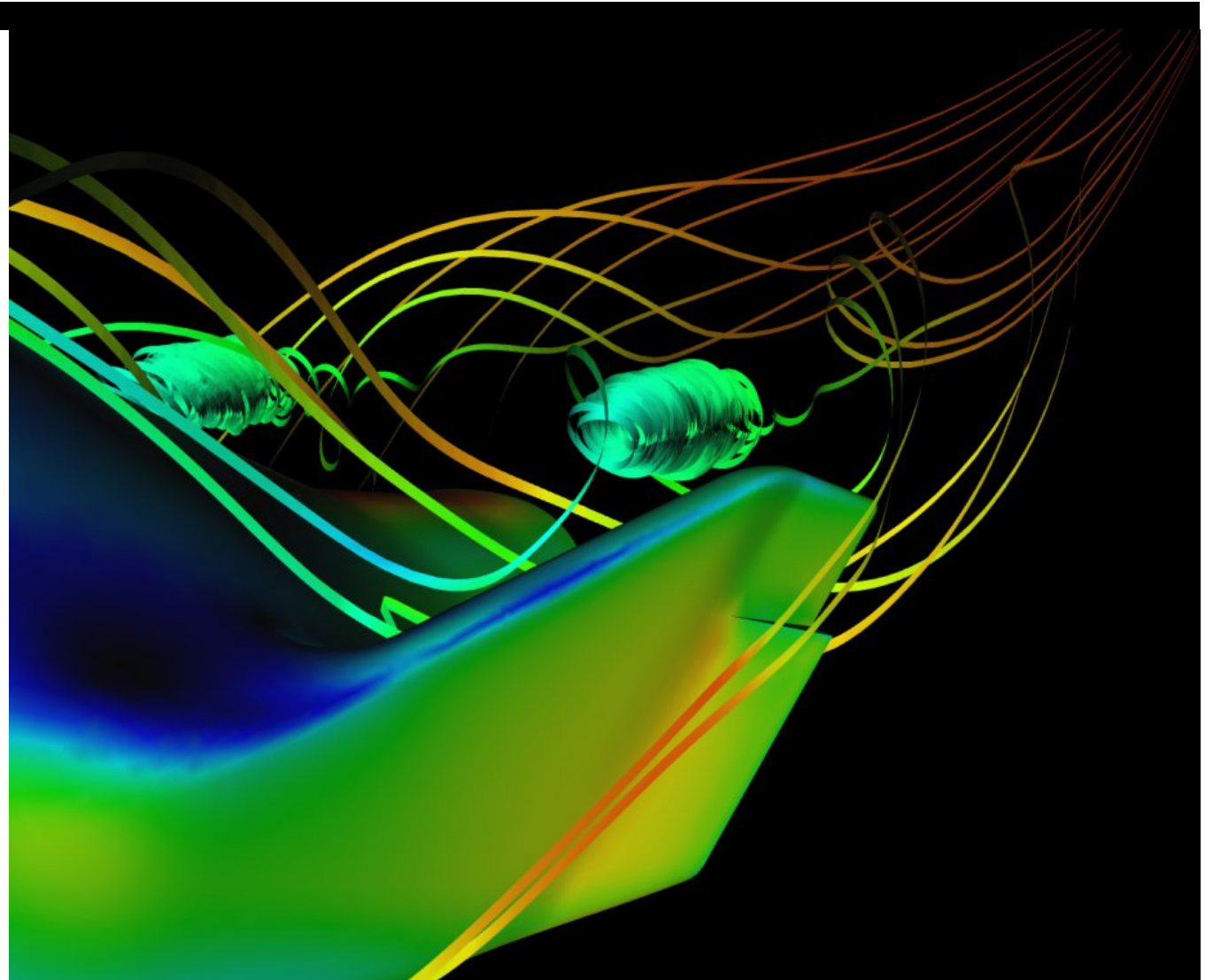
Vector Fields: streamlines

Given a vector field, extract a trace that follows the trajectory defined by the vector.

$$P_{\text{new}} = P_{\text{current}} + V_P \Delta t$$

Streamlines – trace in space

Pathlines – trace in time



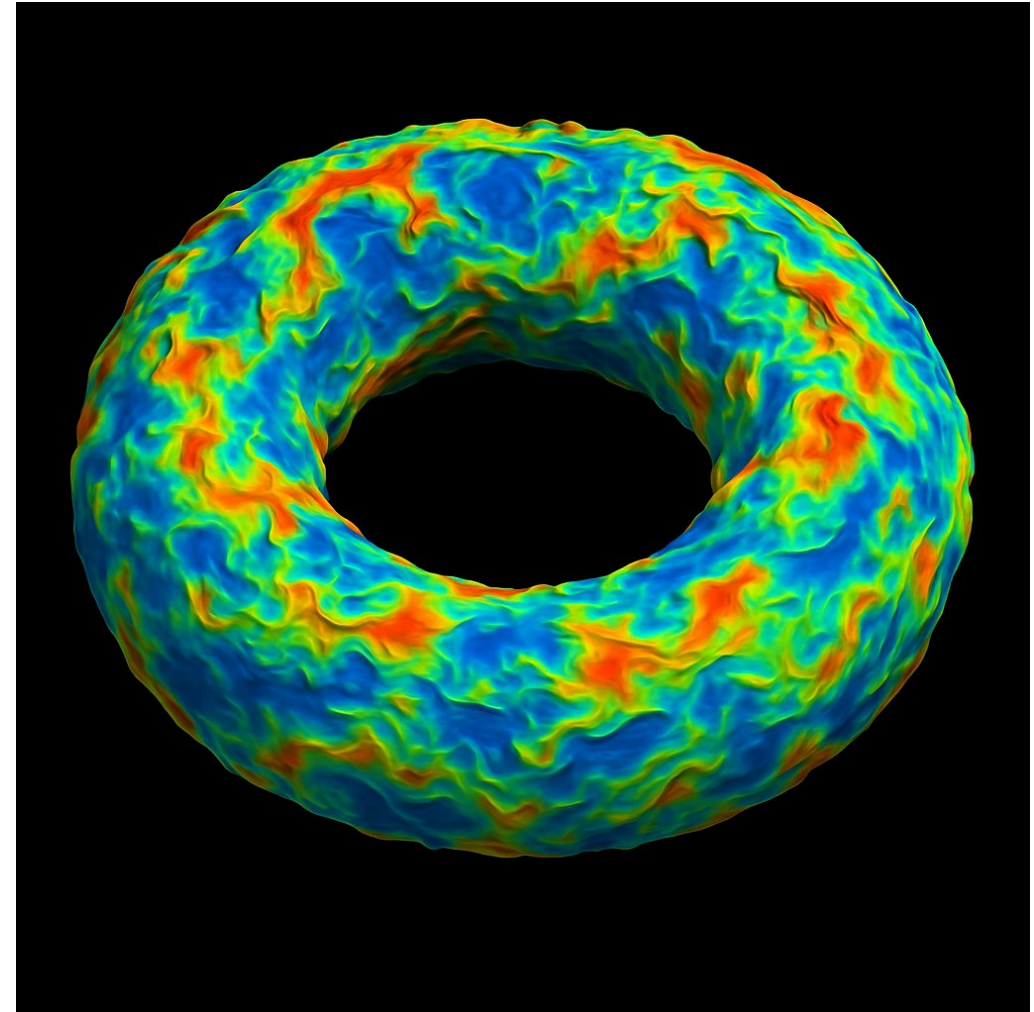


Rendering Techniques (I)

- Surface shading: Pseudo-color
- Surface shading: Global illumination
- Volume rendering: absorption only
- Volume rendering: absorption and emission

Description: an isosurface is extracted and colored using a colormap based on local scalar value or normal direction and transfer function

Features: Simple, fast, color encodes values.



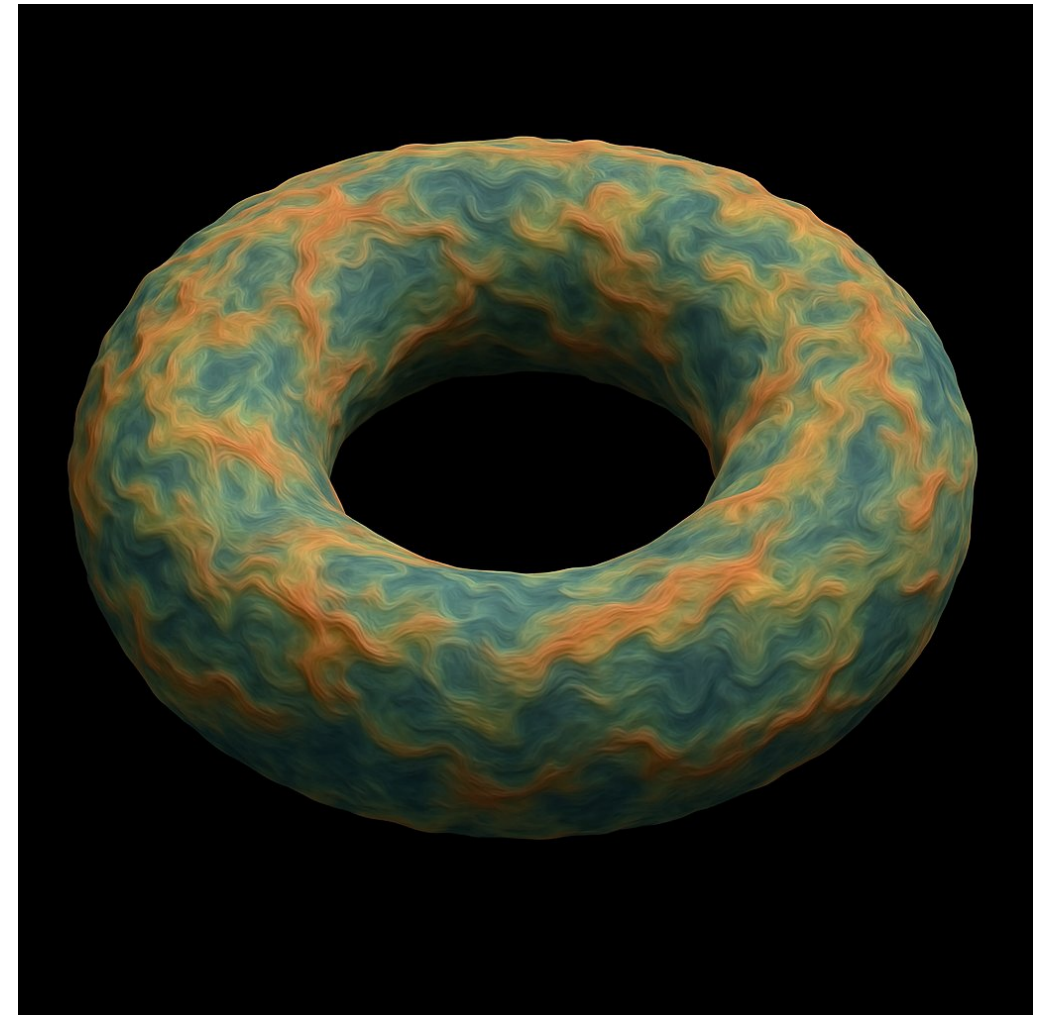


Rendering Techniques (II)

- Surface shading: Pseudo-color
- Surface shading: Global illumination
- Volume rendering: absorption only
- Volume rendering: absorption and emission

Description: Same isosurface, but with realistic lighting: shadows, ambient occlusion, reflections.

Features: Adds realism, shape perception, but more computationally expensive.



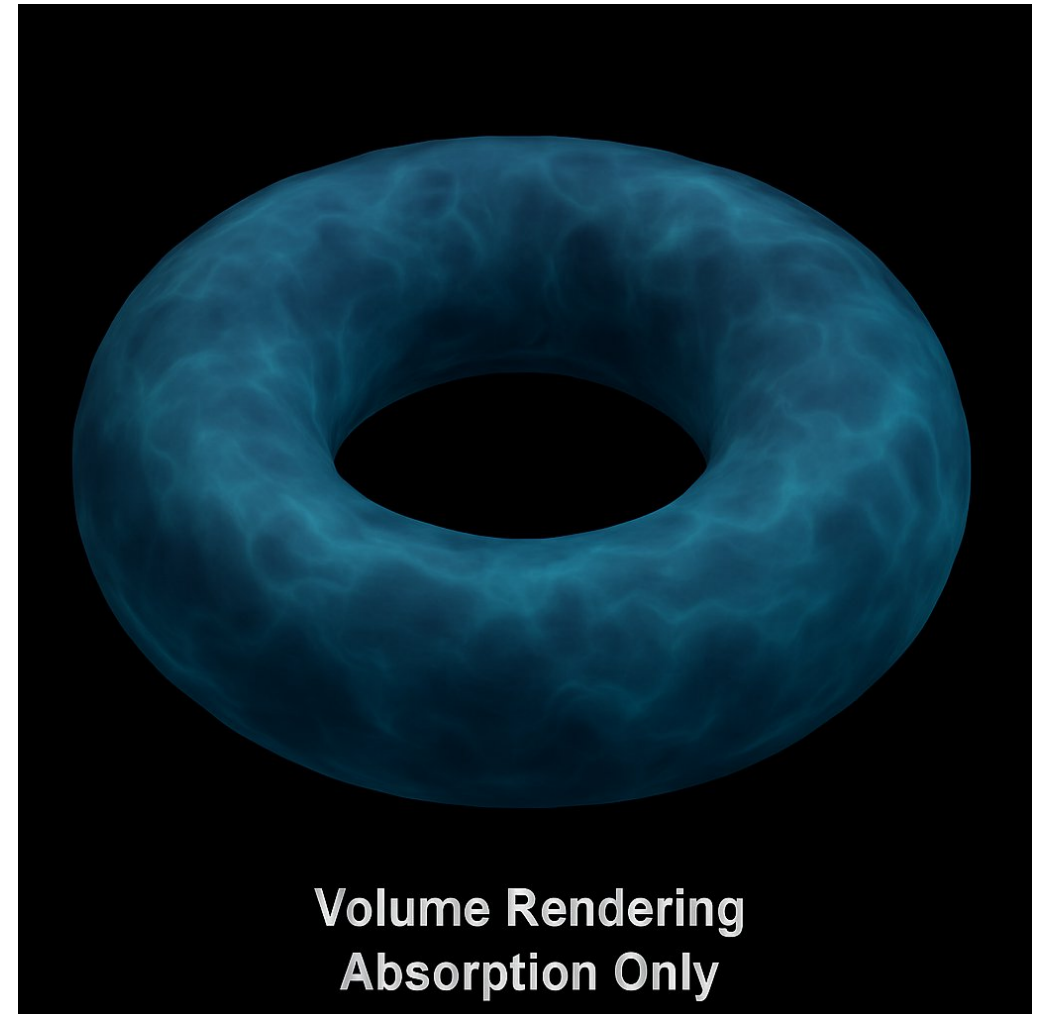


Rendering Techniques (III)

- Surface shading: Pseudo-color
- Surface shading: Global illumination
- Volume rendering: absorption only
- Volume rendering: absorption and emission

Description: Light attenuates exponentially through material (Beer–Lambert law). No scattering, just transparency and density..

Features: Reveals internal structure softly, no depth shading.





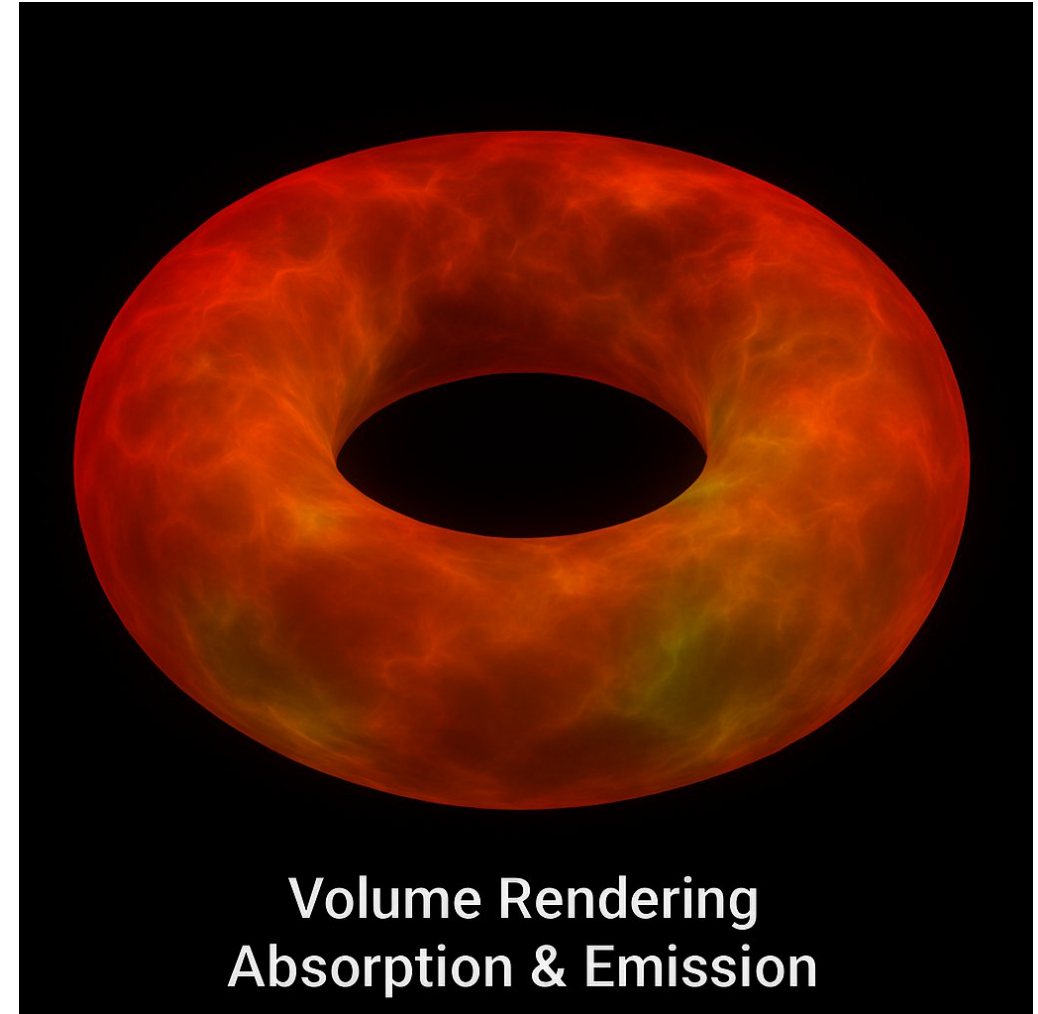
Rendering Techniques (IV)

- Surface shading: Pseudo-color
- Surface shading: Global illumination
- Volume rendering: absorption only
- Volume rendering: absorption and emission

Description: Light attenuates exponentially through material (Beer–Lambert law), but is also added accordingly to the transfer

function

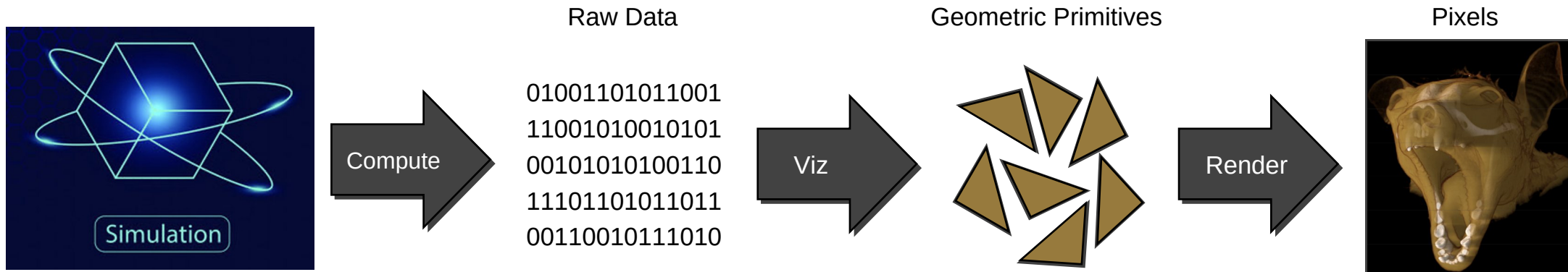
Features: Internal structure becomes more apparent. No scattering.





Going large and distributed...

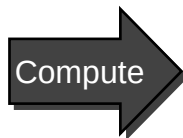
Visualization allows “seeing” the data, gaining insight on its structure and formulating hypotheses about the problem being modelled



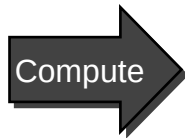


Going large and distributed...

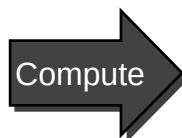
What about distributed simulation and distributed large data sets?



Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010



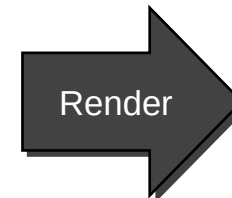
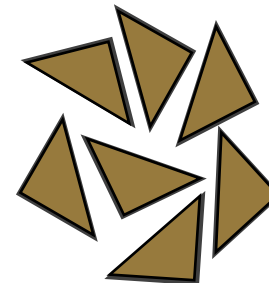
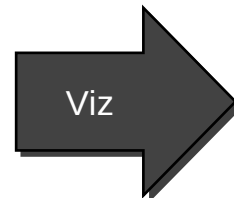
Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010



Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010

Scientific
Visualization

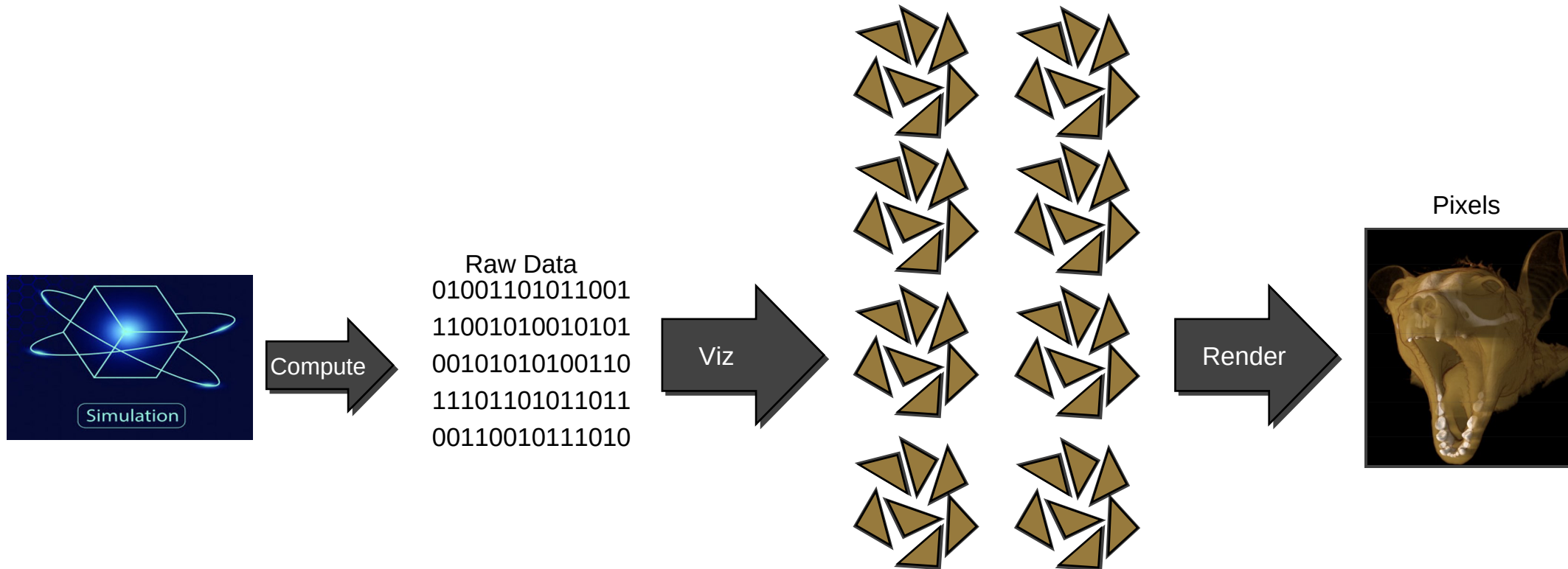
Geometric Primitives





Going large and distributed...

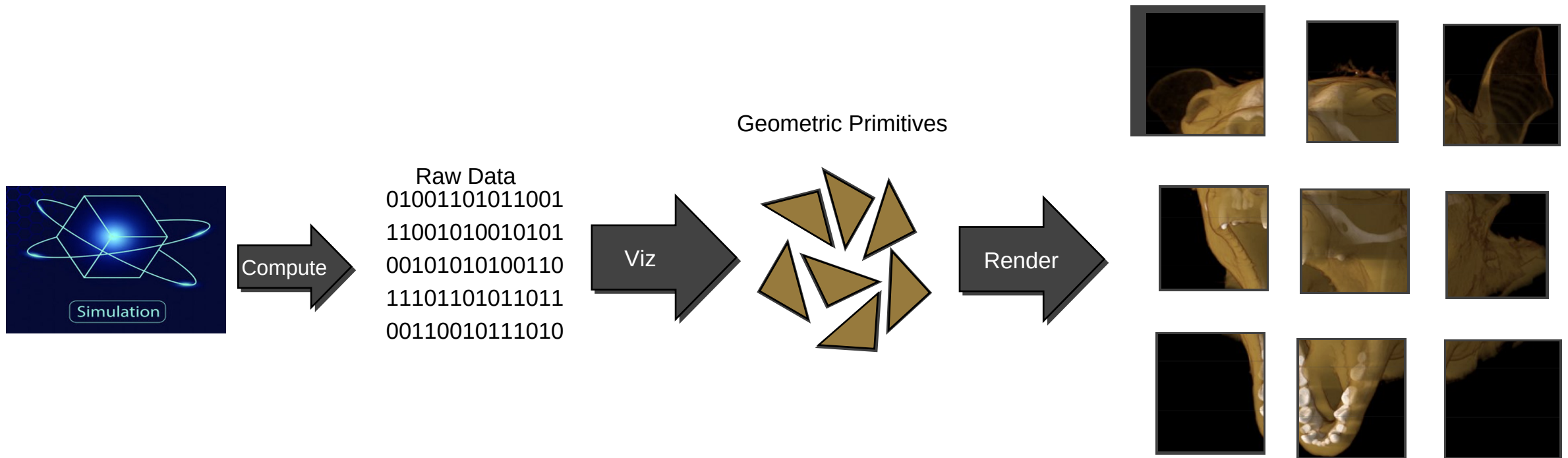
or distributed visualization operations?





Going large and distributed...

or distributed displays?





Going large and distributed...

Or all of them?



Compute

Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010



Compute

Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010

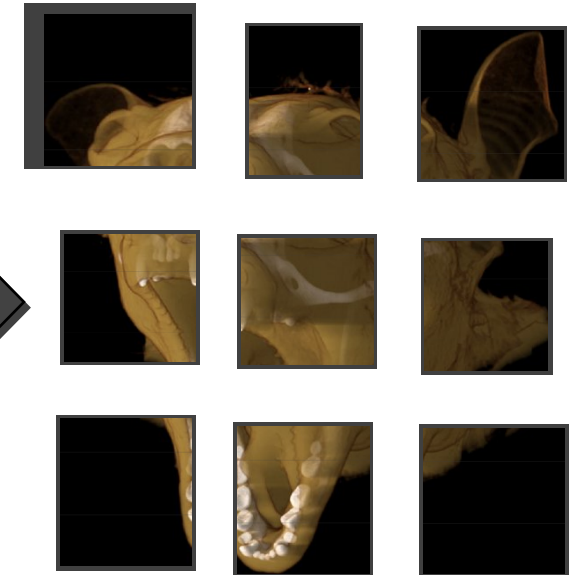
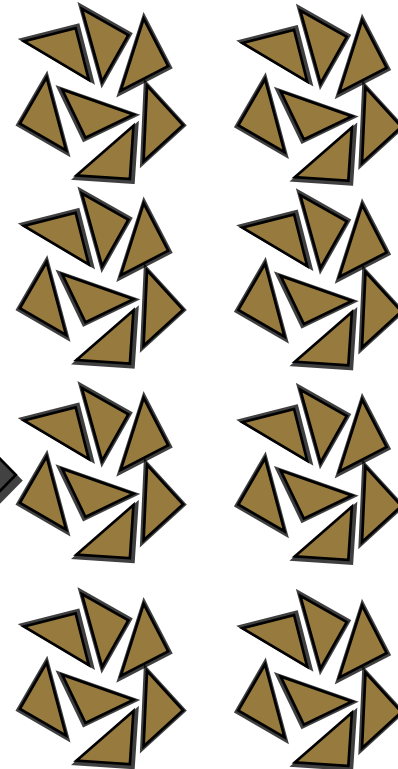


Compute

Raw Data
01001101011001
11001010010101
00101010100110
11101101011011
00110010111010

Viz

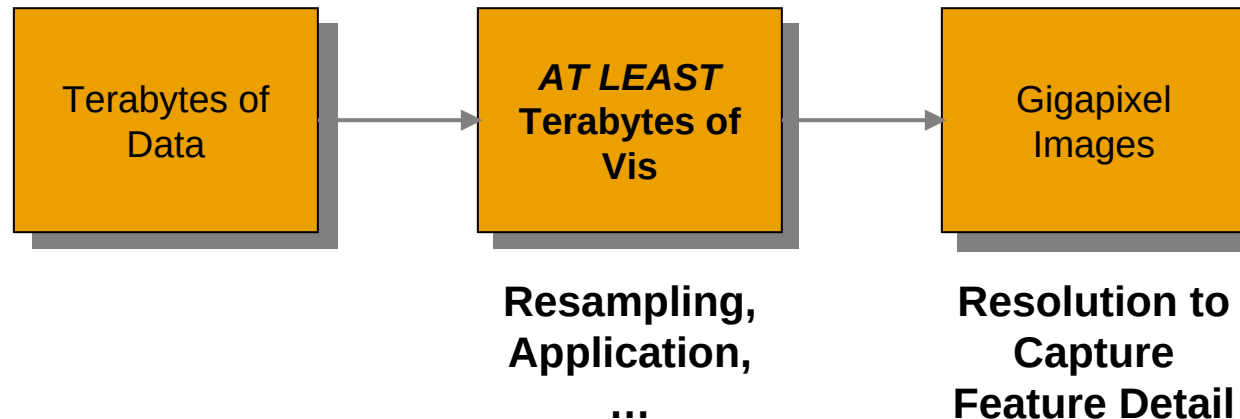
Render





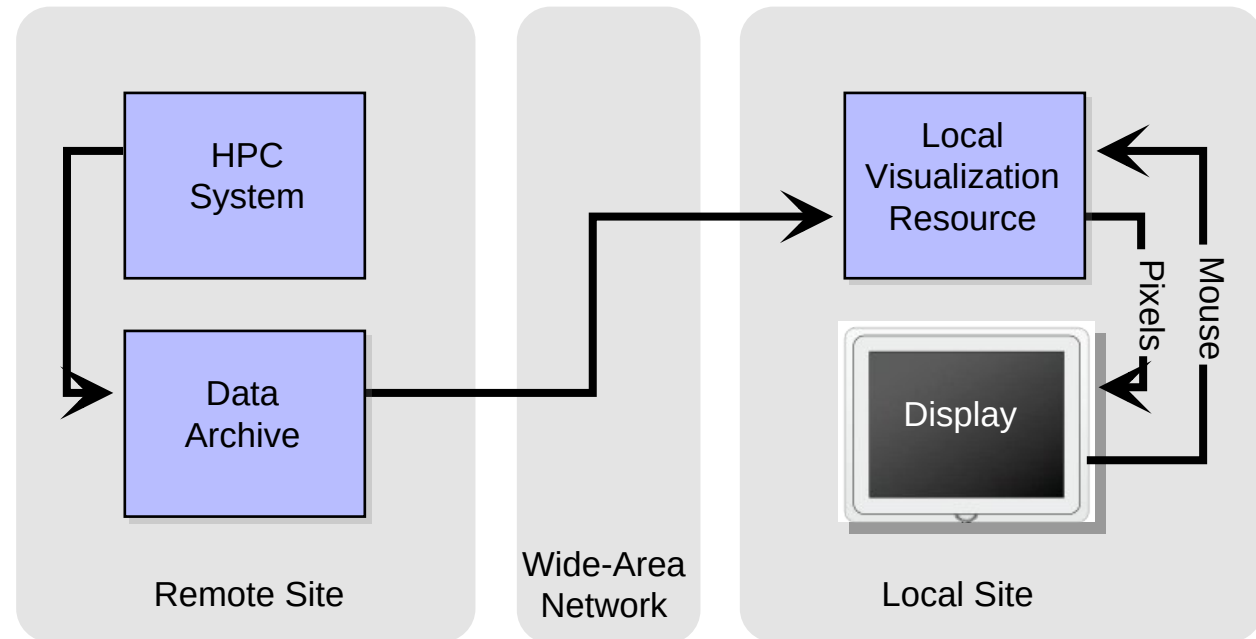
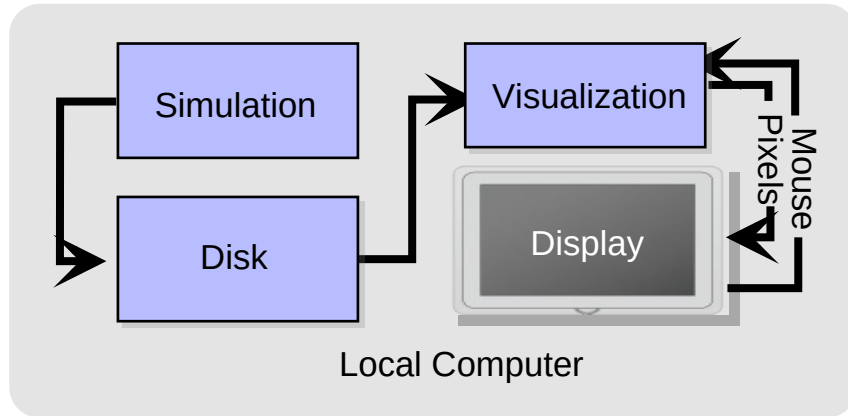
Visualization scalability with HPC

Large data produced by large simulations require large visualization machines and produce large visualization results





Traditional post-processing





How long can you wait?

File Size	10 Gbps	54 Mbps
1 GB	1 sec	2.5 min
1 TB	~17 min	~43 hours
1 PB	~12 days	~5 years

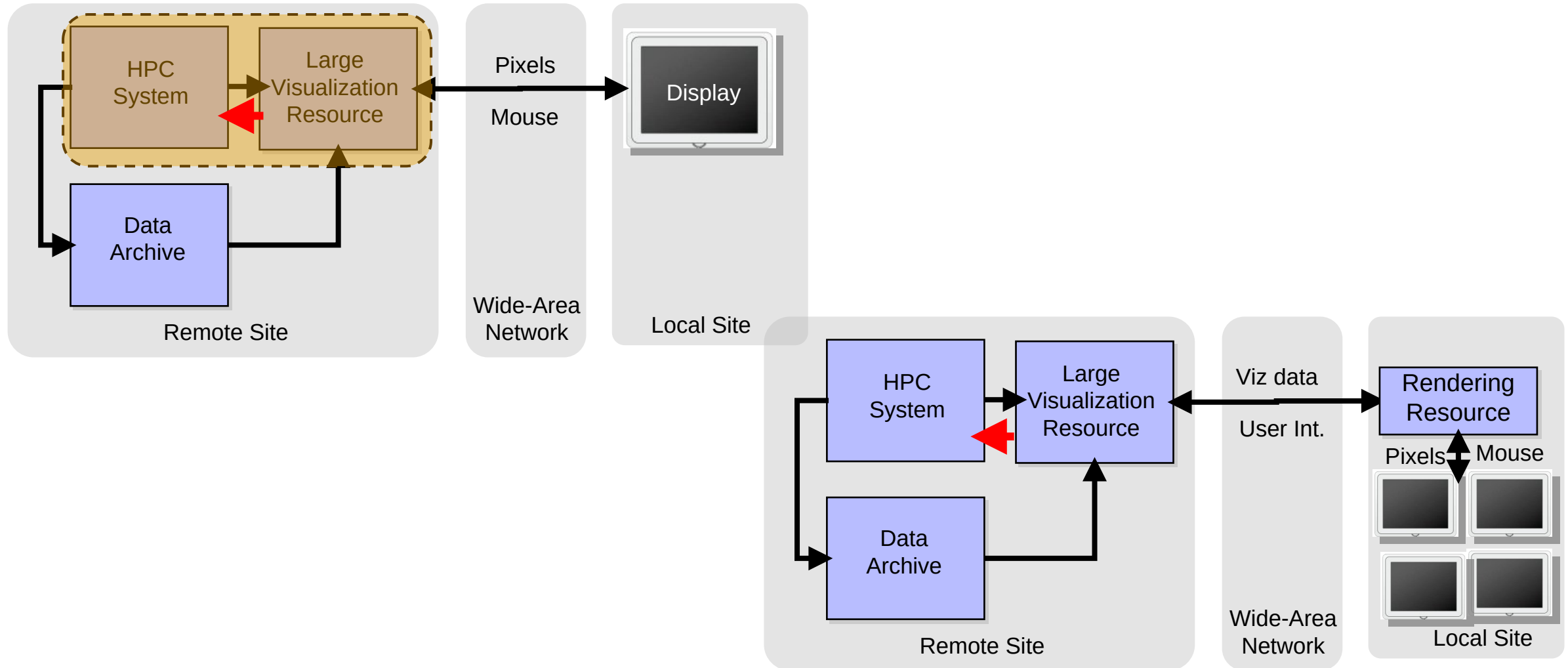
move the Viz computation next to the data : *in-situ*

Viz



Remote *in-situ* visualization

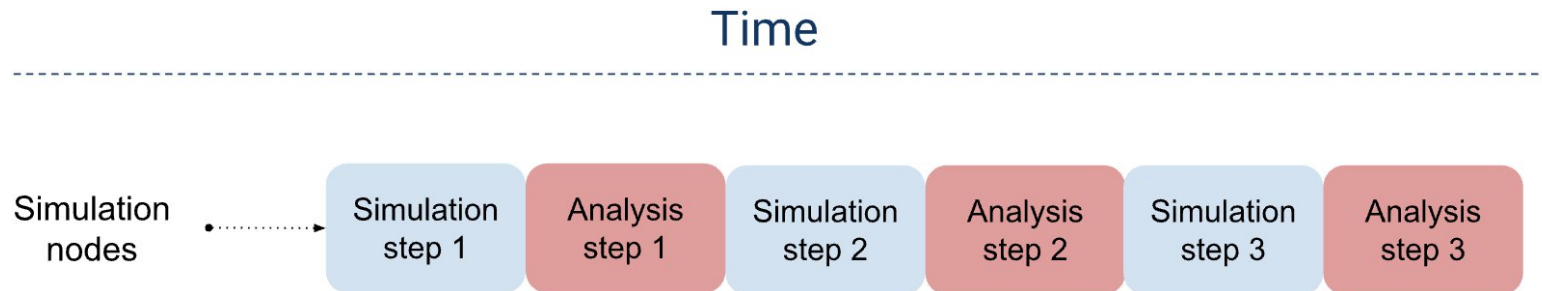
eventually, the same rendering nodes





In situ : generate visualizations from a long-running simulation **after each time step**, instead of waiting for the whole simulation to finish.

Per time step post-processing viz operators, **executed in parallel next to the simulation processes**, generate extracted images or data files

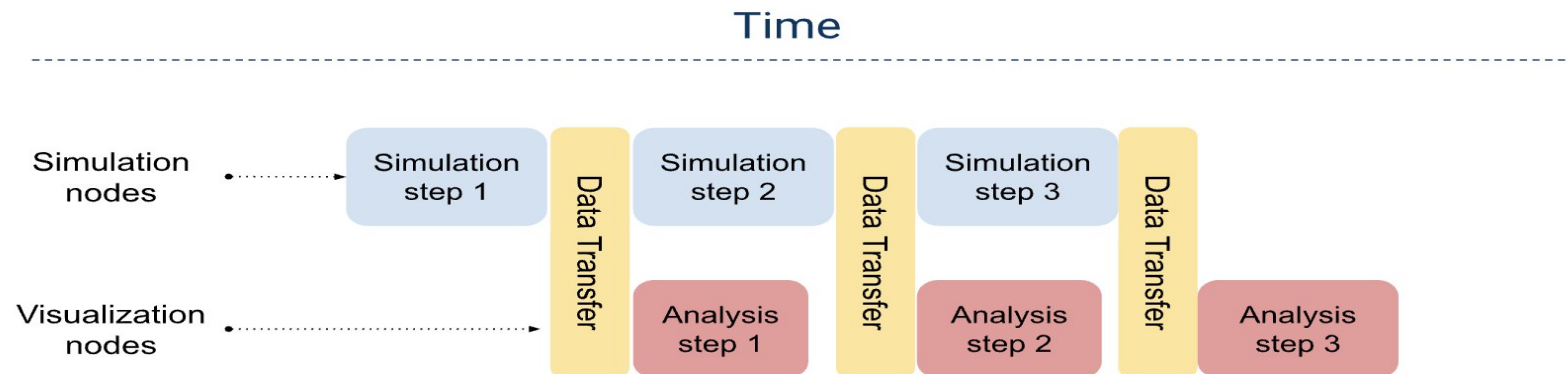


in situ processing happens **synchronously** on the simulation nodes, after each simulation time step.

Supported by [Catalyst-Paraview](#), is referred to as ***in line***



- the visualization pipeline can be delegated to different computation (visualization) nodes
- data is transferred from the simulation resources to the **visualization resources**
- the simulation resumes once the data transfer completes
- the visualization workload is executed **asynchronously**

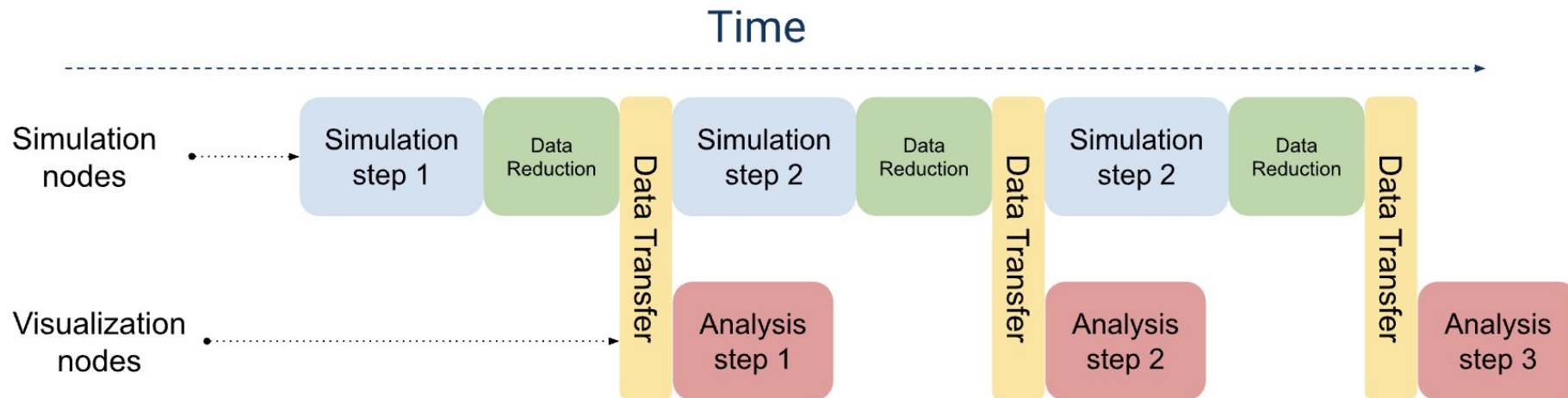


- supported by [Catalyst-ADIOS2](#), is referred to as *in-transit*
- drawback: simulation data is **transferred**, defeating the original purpose of *in situ*.



Hybrid *in situ*, *in transit* Viz

- Large data sets preclude full data transfer among Sim and Viz resources
- *in situ* data reduction on the Sim nodes, then send the data *in transit* to the visualization nodes





Galaxy Evolution

[NASA Scientific Visualization Gallery]